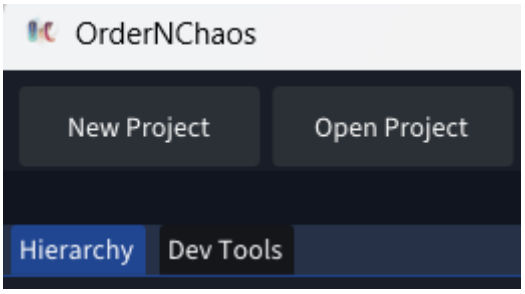
editorGuide

1.0 Opening Project **3**
1.1 Creating New Project 3
2.0 Hierarchy Tab **4**
2.1 Creating Entities 4
2.2 Creating Parent-Child Relationships 4
2.3 Deleting Entities 5
2.4 Duplicating Entities 5
2.5 Showing Entity IDs 6
2.6 Loading, Editing, and Saving Scenes 6
3.0 Old Asset Manager Tab **9**
3.1 Refresh Asset Lists 10
3.2 Drag and drop assetID into Inspector render field 10
3.3 Selectable Scenes 10
3.4 Creating a new Scene file 10
3.5 Asset Drag and Drop from file explorer 10
3.6 Error Detection 11
3.6 Duplication Detection 12
4.0 Asset Manager Tab **13**
4.1 Overview 13
4.2 Visual Asset Browser & Thumbnails 13
4.3 Folder Management 13
4.4 Drag & Drop Asset Organisation 14
5.0 Console Tab **15**
6.0 Inspector Tab **16**
6.1 Adding Component 16
6.2 Parent-Child Relationships 17
6.3 Remove Components 17
6.4 About Components 18
7.0 PrefabInspector Tab **26**
7.1 Editing Prefabs 26
7.2 Dev Tools - Prefab Editor 26
8.0 Animation Tab / SpriteSheet Splicer Tab **27**
8.1 About Animation (In Animation Tab) 27
8.2 Spritesheet Slicer Window (Preview Panel) 28
8.3 Grid Configuration 29
8.4 Animation States 29
8.5 Exporting Animation (.gganim) 30
8.6 Loading Existing .gganim Files 31
9.0 Tilemap Designer Tab **32**
8.1 Overview 32
9.2 File Menu (New / Open / Save / Save As) (At the top of the panel) 33
9.3 Tools (Select / Paint / Erase / Fill) 33

9.4 Tile Palette Panel (Left) 35
9.5 Properties Panel (Right) 36
10.0 Editor view / Game view **36**
10.1 Play / Pause / Stop / Resume Toolbar 37
10.2 Gizmos 38
10.3 Camera 41
11.0 Layer Manager Tab **42**
10.1 Layer Types 42
11.2 Enabling and Disabling Layers 43
11.3 Batch Layer Operations 43
11.4 Defining Layers in the Inspector Tab 43
12.0 Build Size Analyzer **44**
13.0 Asset Used Report **45**
14.0 Others **46**
14.1 Docking 46
14.2 Lua/Scripting 47
14.3 Undo and Redo 47
14.4 FullScreen & Window mode 47
14.5 Show all entities on screen 47

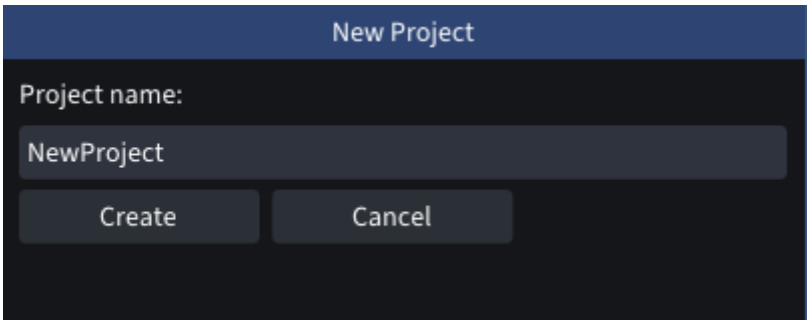
1.0 Opening Project

Overview



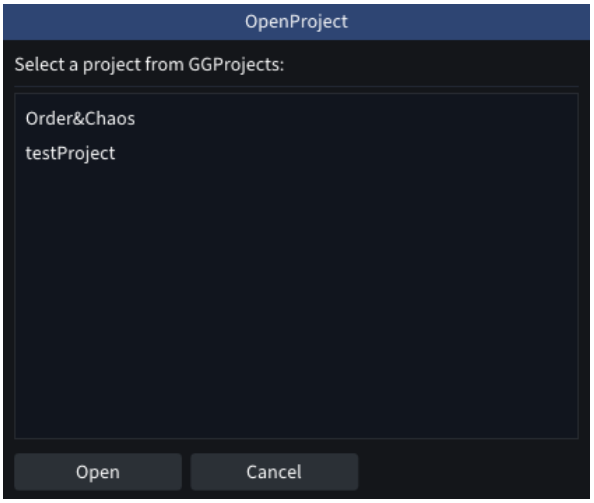
The Project buttons are rendered at the top left of the game engine.
The Project buttons are there to manage different game projects inside a runtime game engine. It has “New Project” to create new project, and “Open Project” to open existing projects.

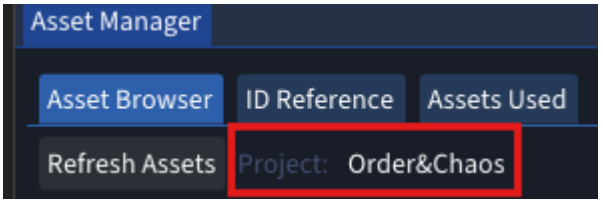
1.1 Creating New Project



- Click the “New Project” Button, it will open a pop up to allow you to enter your new project name.

1.2 Opening Existing Project





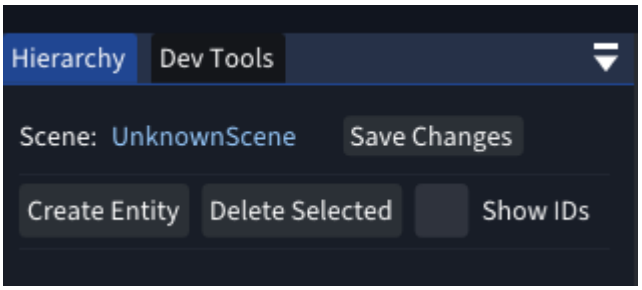
- Click the “Order&Chaos” Button, it will open a pop up to allow you to open your existing projects, and the Current open project will be displayed at Asset Manager.

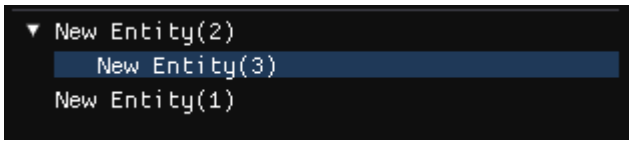
2.0 Hierarchy Tab

Overview
The Hierarchy Tab is dockable on the left side by default.

The Hierarchy Node Panel in the editor serves as the interface for managing entities within a scene. It provides a tree-view structure, allowing to visualise, create, duplicate, and rearrange entities (including parent-child relationships). When an entity is selected, its properties can be viewed and modified in the Inspector tab.

2.1 Creating Entities

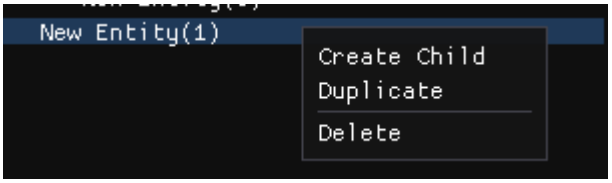


- Click the “Create Entity” Button at the top of the panel.
    - If no entity is selected in the panel, the new entity appears at the root level (click on any area in the Hierarchy tab to deselect objects before creating).
    - If an entity is selected, the new entity becomes its child
    - 

2.2 Creating Parent-Child Relationships

Parent-Child hierarchies allow linking (moving/scaling a parent affects its children in the Inspector Tab).

Method 1 - Right-Click Menu

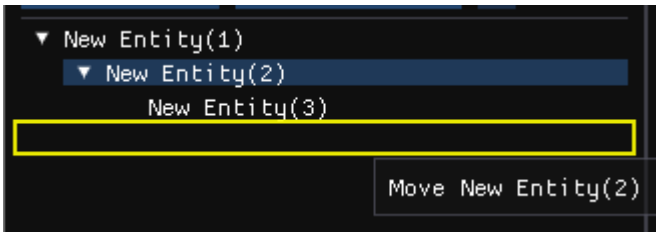


- Right-click on the selected entity (this becomes the parent).
- Choose “Create Child”.
- A new child entity appears nested under the parent.

Method 2 - Drag and Drop

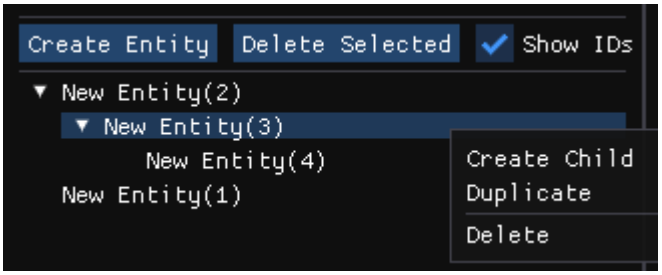


- Left-click to drag an entity from a list.
- Drop it onto another entity to set it as a child (can drag both parent and/or child and set it as a child under another parent)



- To make a root entity, drag it onto the highlighted empty space under the hierarchy tree.

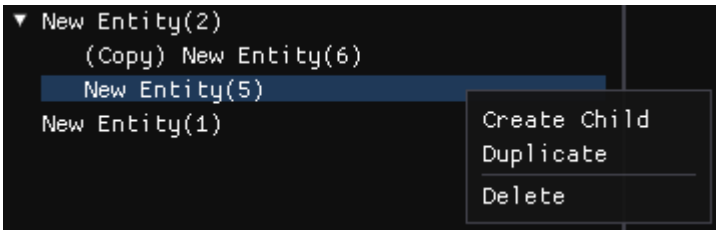
2.3 Deleting Entities



- Select an entity
- Click the “Delete Selected” button or right-click -> Delete
    - 

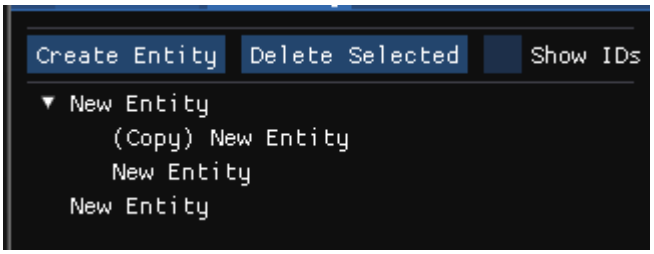
    - The entity and all its children are removed from the world recursively

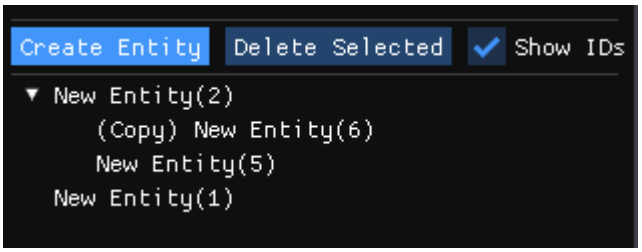
2.4 Duplicating Entities



- Right-click an entity and select “Duplicate”.
- This creates a copy of the entity
- The duplicate’s name is prefixed with “(Copy) “

2.5 Showing Entity IDs

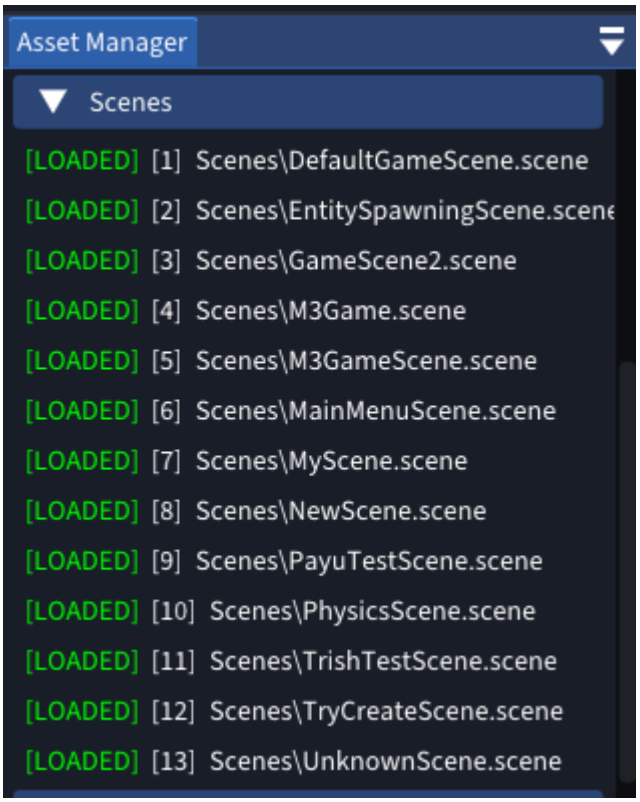




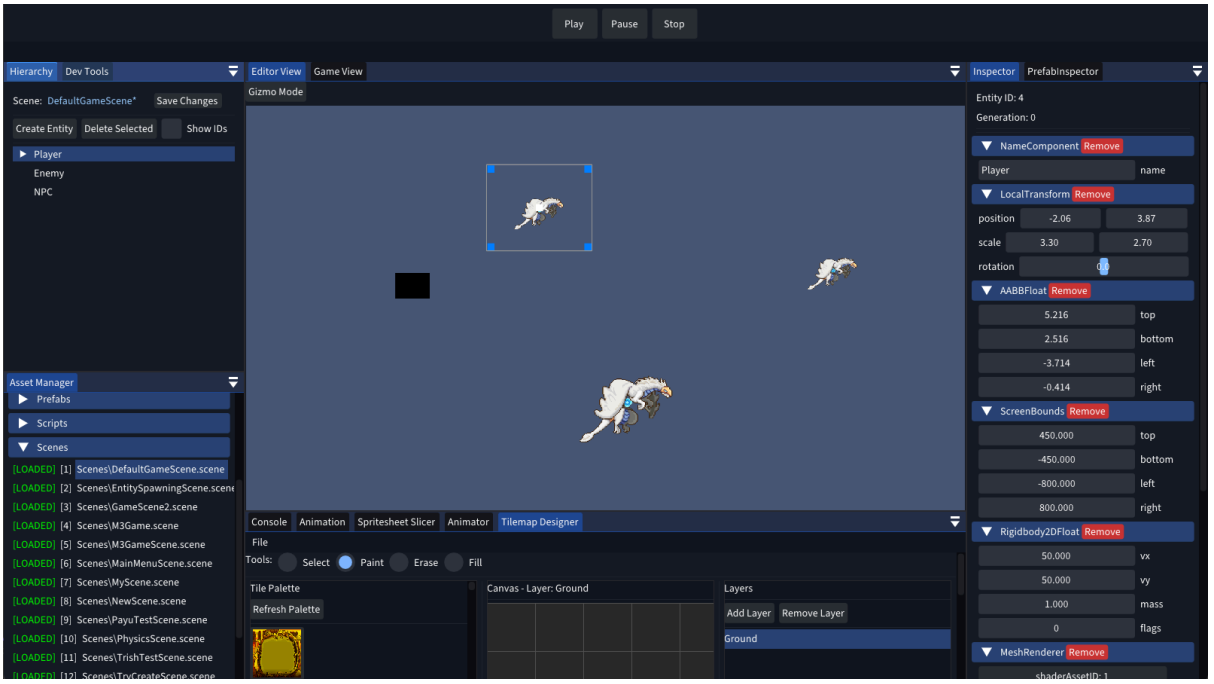
- Enable the “Show IDs” checkbox to display each entity ID beside its name (Useful for debugging).

2.6 Loading, Editing, and Saving Scenes

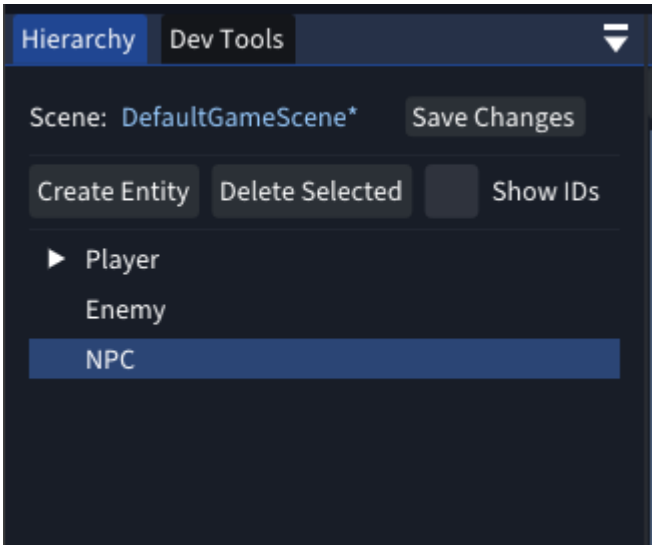
- The user can utilise the AssetManager to launch a loaded scene. Simply double-click any loaded scene in the Asset Manager to launch it.



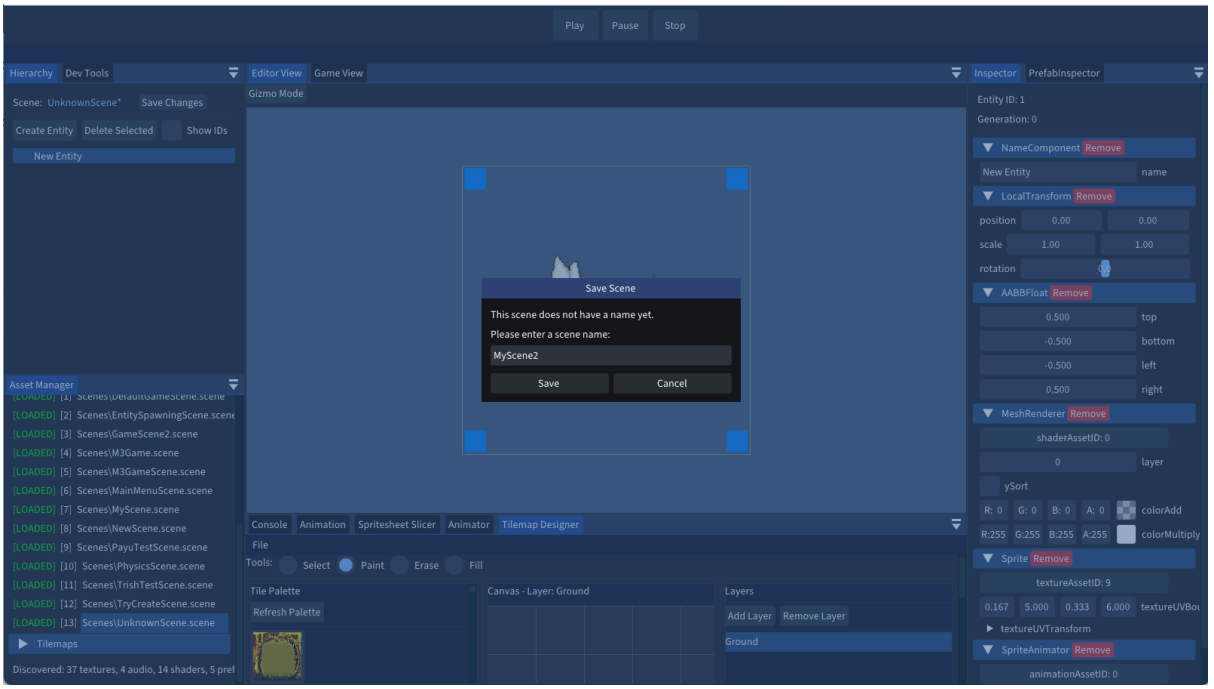
- Once a scene has been instantiated, the Hierarchy node will show the scene’s object list.



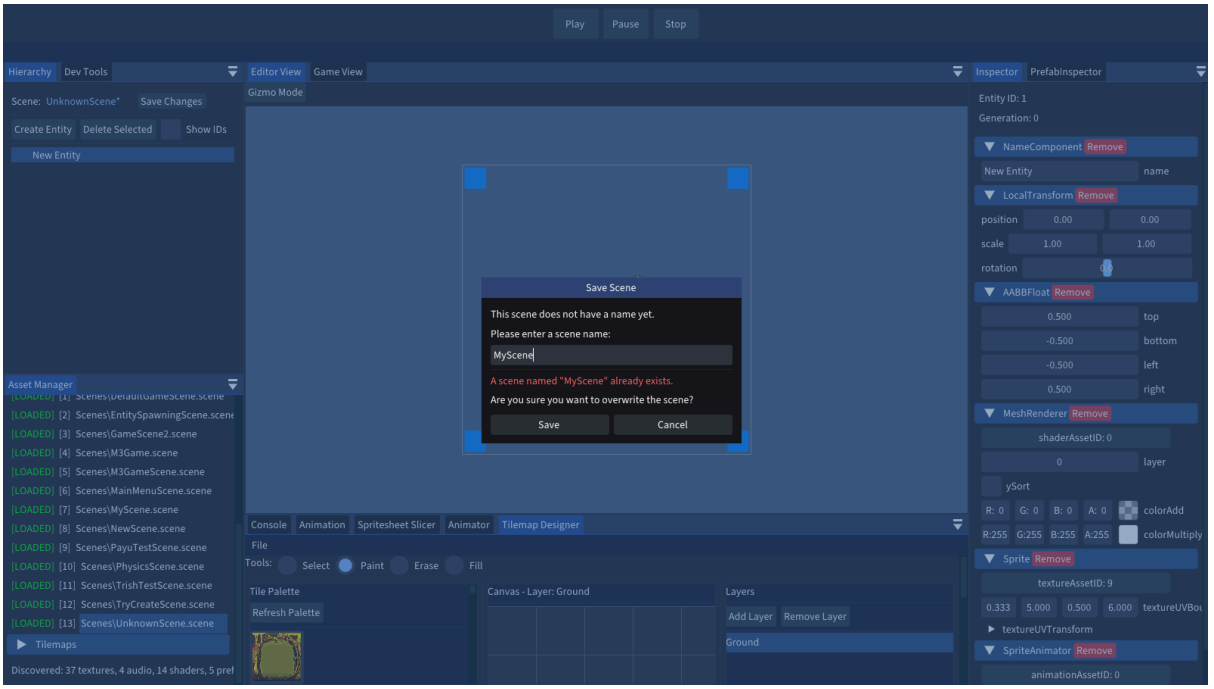
- The user can use the Hierarchy node to spawn new entities, and edit entities using the Inspector node.
- Once done, the user can simply press Ctrl+S to save the scene. Modified scenes will be marked dirty with an asterisk (*) to notify the user that the scene is unsaved. A “Save Changes” button will also appear beside the scene name.



- If it is a new scene (UnknownScene), a pop-up window will appear to prompt the user to save the scene under a new name. By default, the editor will suggest the name MyScene with a number suffix to prevent accidentally overwriting existing scenes.

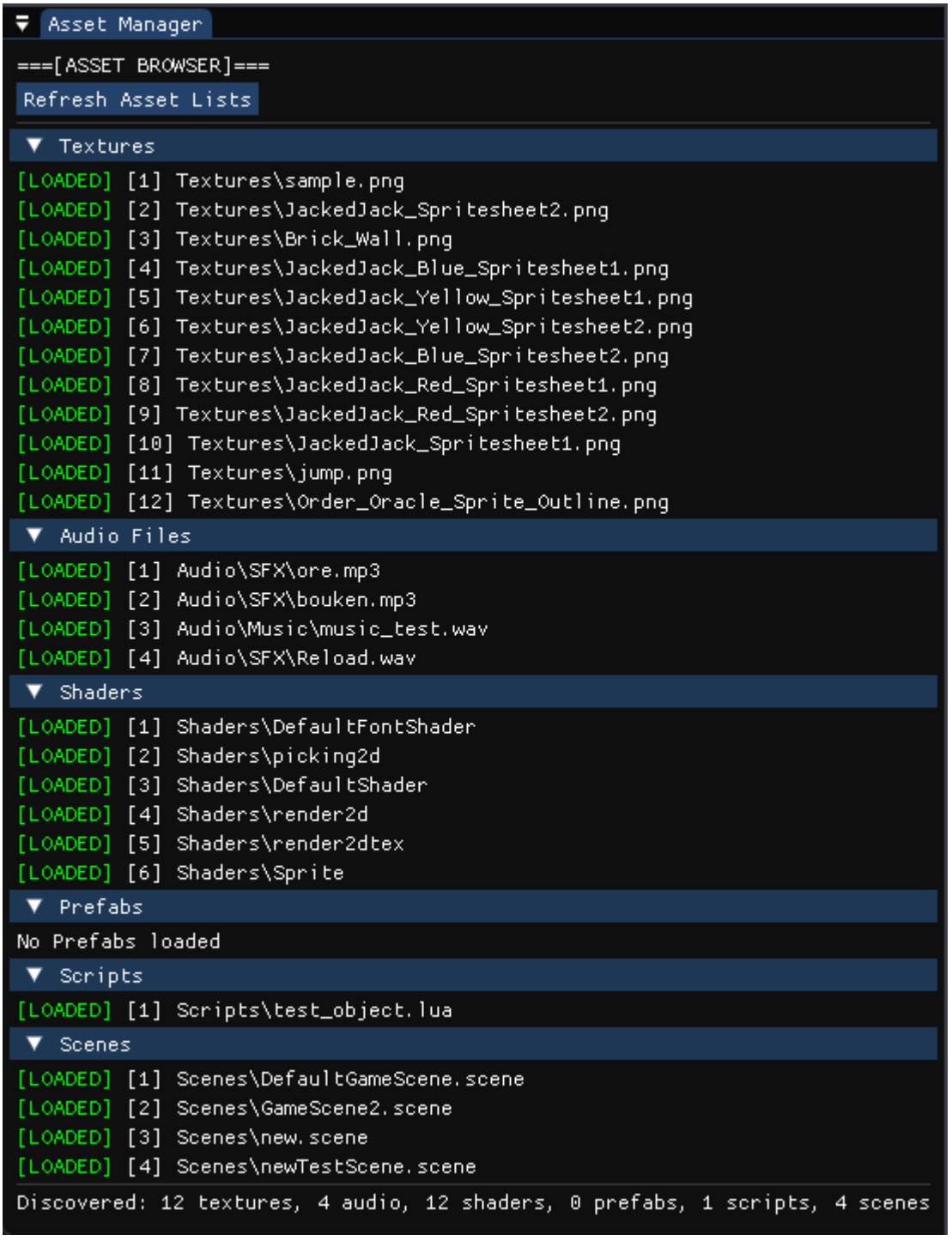


- If the user proceeds to save the scene under an existing name, the editor will also warn the user that they are overwriting an existing scene.



3.0 Old Asset Manager Tab

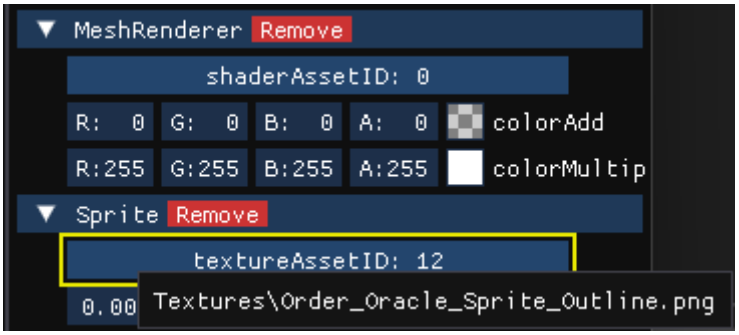
Asset Browser:



3.1 Refresh Asset Lists

- "Refresh Asset Lists" button will display newly added assets via adding assets through window explorer "Assets" folder

3.2 Drag and drop assetID into Inspector render field

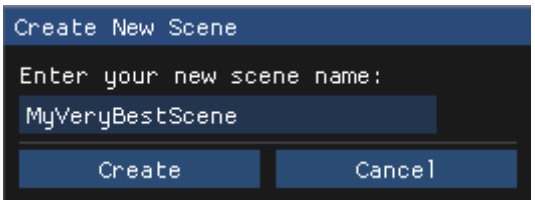


- Every different Asset has its own assetID. They are selectable and can be dragged and dropped into their specific asset render field in the Inspector window.

3.3 Selectable Scenes

- Double-clicking on a "Scene" file will render/display its saved content into the game viewport.

3.4 Creating a new Scene file



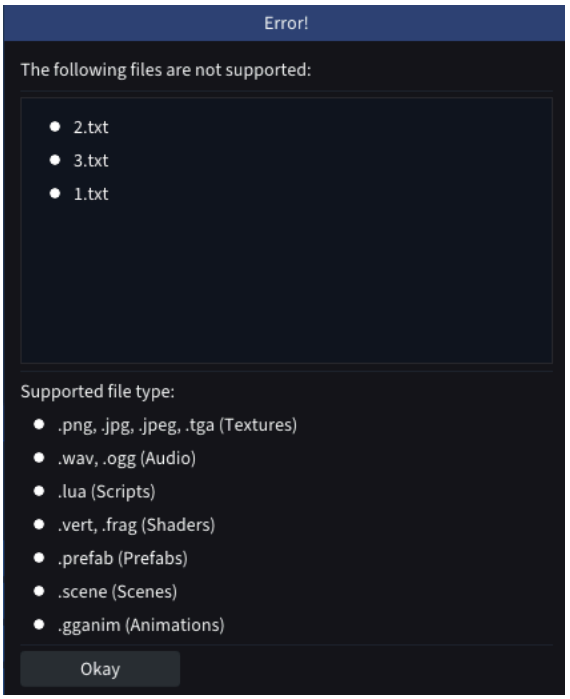
- Create a new "Scene" asset by right-clicking in the "Asset Browser" window and selecting the "Create New Scene" option.
  Enter the name you want for the "Scene" asset, and your newly named "Scene" asset will appear under the "Scene Header" instantly with the help of "hot reloading".

3.5 Asset Drag and Drop from file explorer

- Able to handle multiple different asset files in one drop.
- Drag supported asset files from File Explorer and drop them into the Game Window. The asset files will appear in the "Asset Browser" window in their specific header and be ready for use.

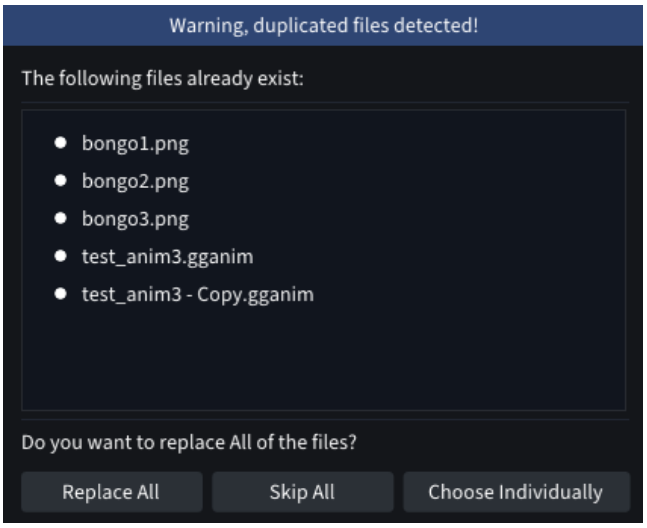
Supported Asset files list:
Texture: png, jpg, jpeg, tga
Audio: wav, ogg
Script: lua
Shader: vert, frag
Prefab: scene
Animation: gganim
tilemap: tilemap

3.6 Error Detection

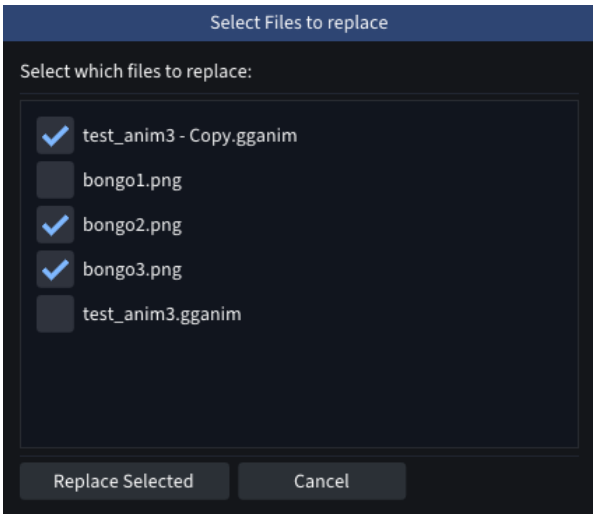


- Detects every file that is not supported and triggers a popup to display every unsupported file.

3.6 Duplication Detection



- Detects every file that has an existing file name in the asset folder and triggers a popup to display every duplicated file.
- Able to Replace All, Skip All or Choose individually



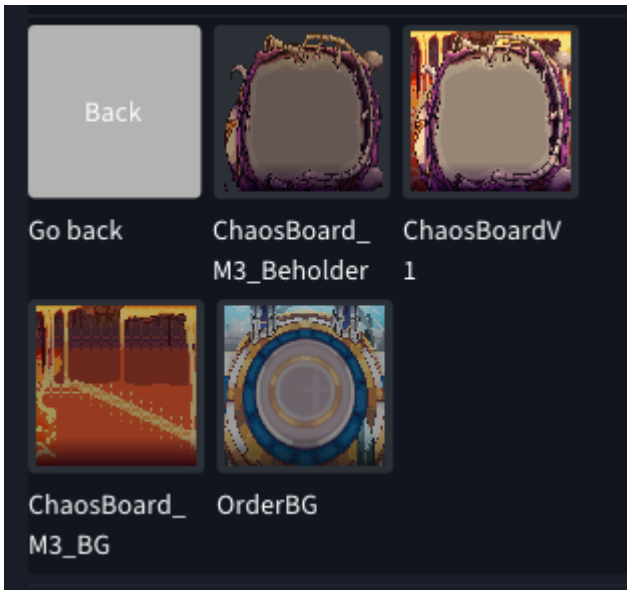
- Choose Individually will be able to replace every checked file or Cancel.

4.0 Asset Manager Tab

4.1 Overview

The New AssetManager is an updated asset browser, introducing navigation, thumbnail previews, and enhanced drag-and-drop workflows while maintaining compatibility with existing editor behaviour (**The old AssetManager (renamed to ID reference) remains available under a separate tab**).

4.2 Visual Asset Browser & Thumbnails

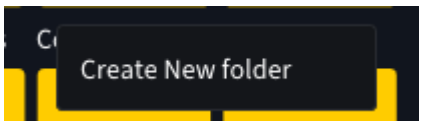
- Textures are displayed with thumbnail previews
  - 

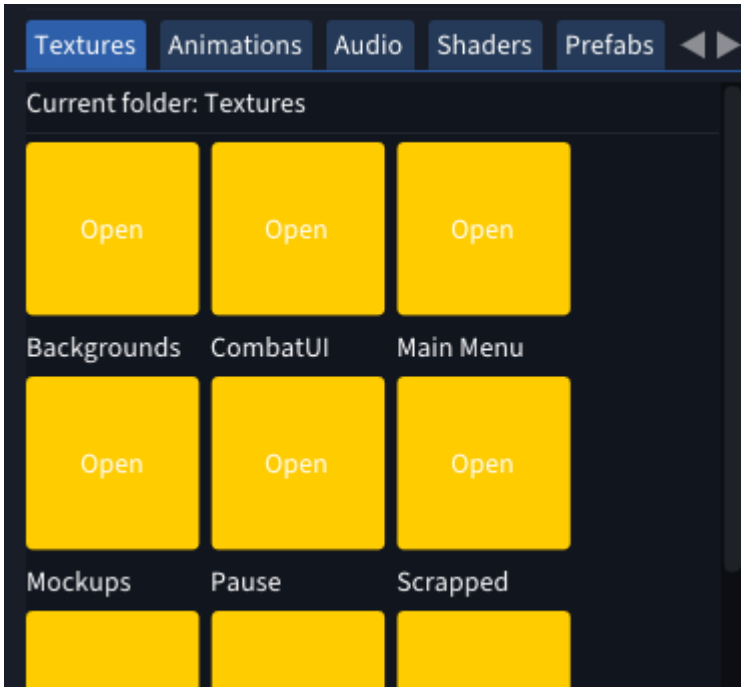
- Non-texture assets currently display a black background thumbnails
  - 

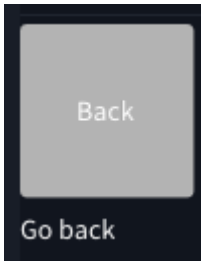
- Improves visual identification and browsing speed

4.3 Folder Management

- Users can create new folders directly in the Asset Browser by right-clicking

- 

- Sub-folders are supported
- Folders can be renamed
- Double-click a folder to open and view its contents
  - 

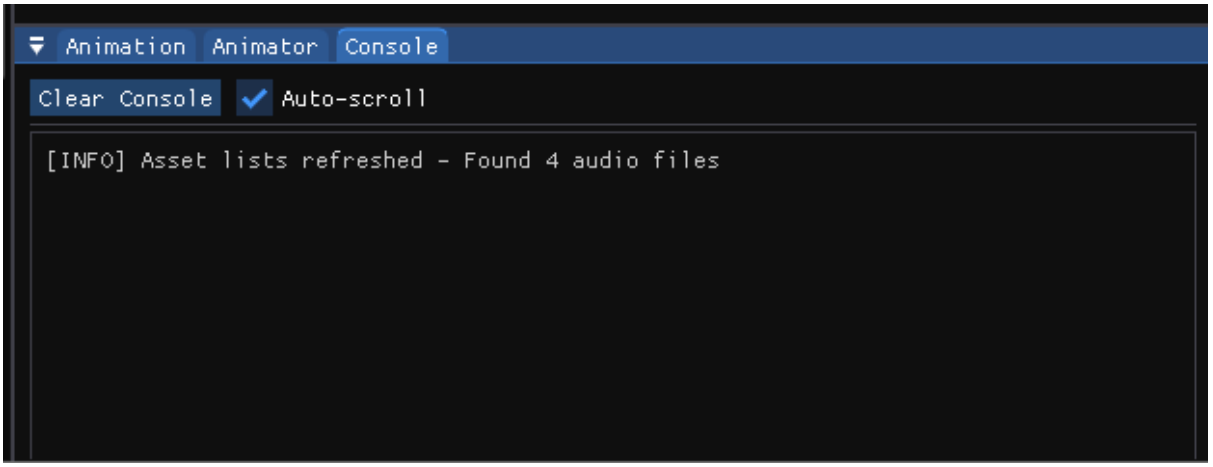
- A Back button allows navigation to the parent folder
  - 

4.4 Drag & Drop Asset Organisation

- Assets can be dragged and dropped into folders within the Asset Browser
- Dragging an asset onto the Back button moves it to the parent folder
- Supports single-asset drag and drop (multi-select currently not supported)

5.0 Console Tab

Provides real-time logging output from the engine and editor systems. Any informational messages, warnings, and error logs generated during editing or gameplay (Messages are printed out using GGLogger).



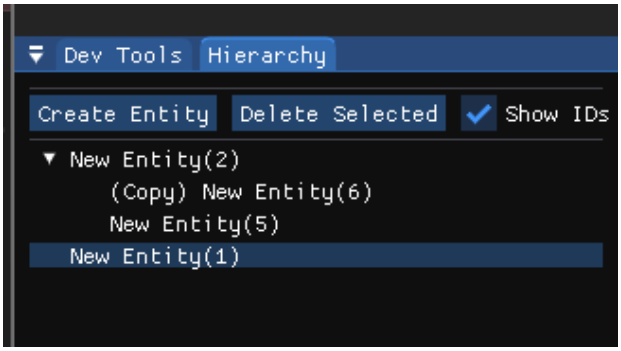
- Clear console:
    - Click the “Clear Console” button to remove all current log entries from view.
- Auto-scroll:
    - When checked, the console automatically scrolls to the newest message.

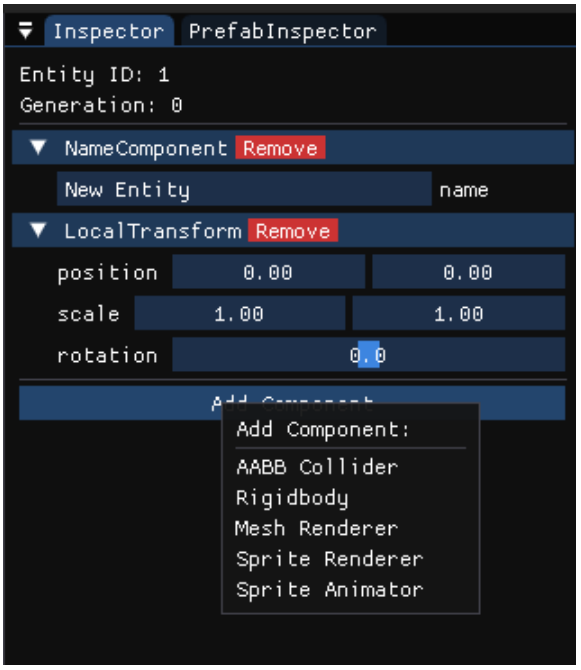
6.0 Inspector Tab

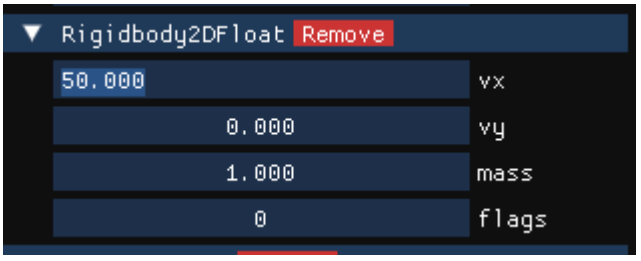
Overview
The Inspector Tab is dockable on the right side by default.

Shows and allows editing of the selected entity's components. Enables the addition or modification of component values in real-time.

6.1 Adding Component





- Select the entity that you want in the "Hierarchy" tab. It lists all the components attached to the entity.
    - If it's a new entity, the NameComponent and Local Transform are given by default.
- To add an entity, left-click on the "Add Component" Button and choose the component that you want (E.g. AABB Collider, Rigidbody, Mesh Renderer, etc).
- 

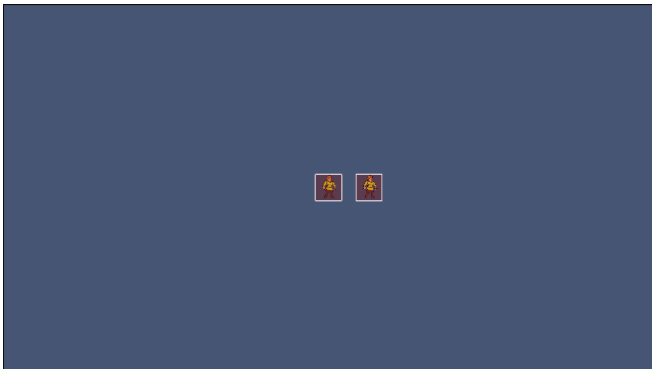
    - To edit values, double-click on the sub-component that you want to change.
- 

    - For sub-components "colorAdd" or "colorMultiply", a color picker is available by left-clicking the square beside them.
- 

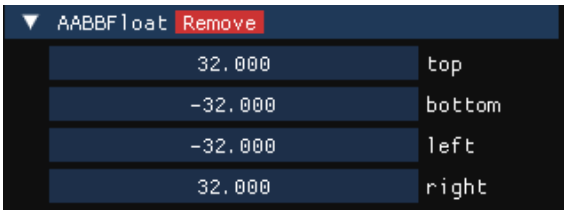
    - For rotation, a slider is available for changing values.

6.2 Parent-Child Relationships

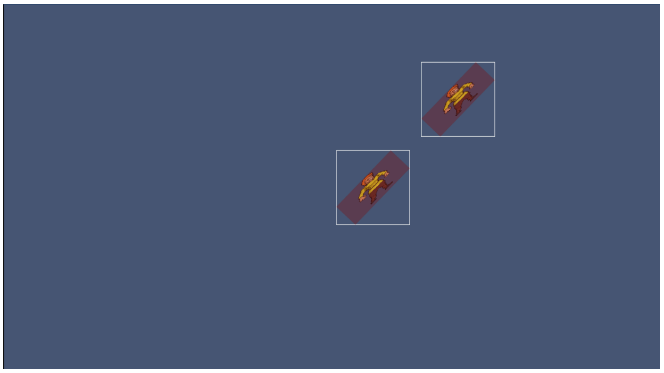
- If the Transform, Rotation, Scale (TRS) of a parent changes, the TRS of the child also changes and AABBFloat will be recalculated accordingly to the scale, as shown below:
- Before

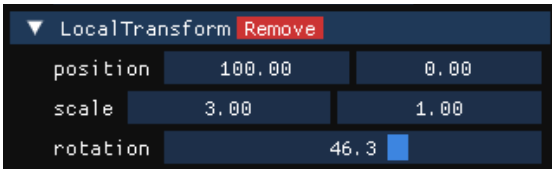






- After







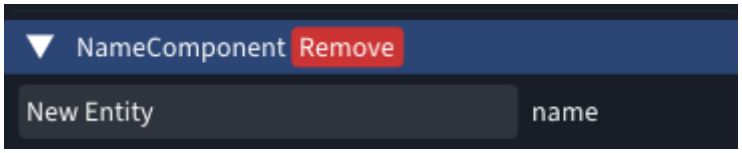
6.3 Remove Components

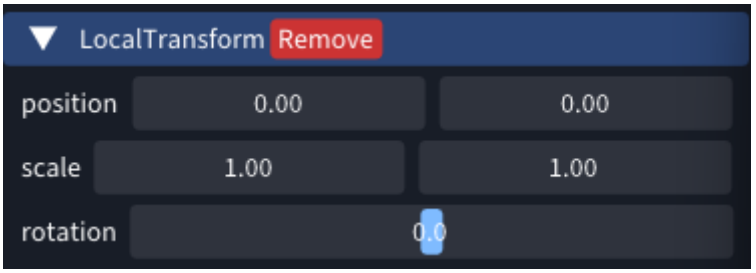


- Remove Components by clicking on the “Remove” Button.

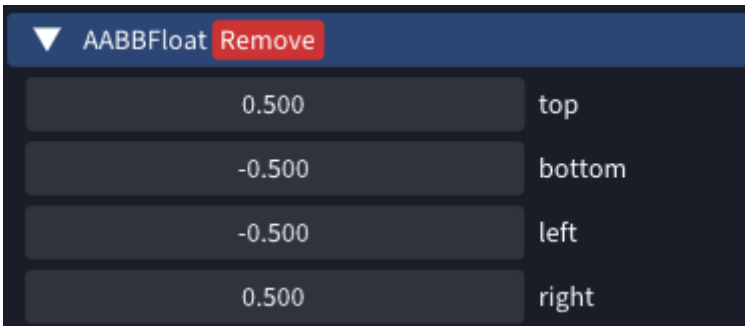
6.4 About Components

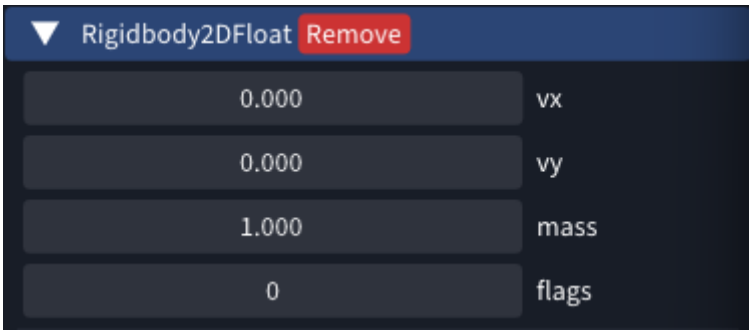


- ***NameComponent (Default Given)***
  - 

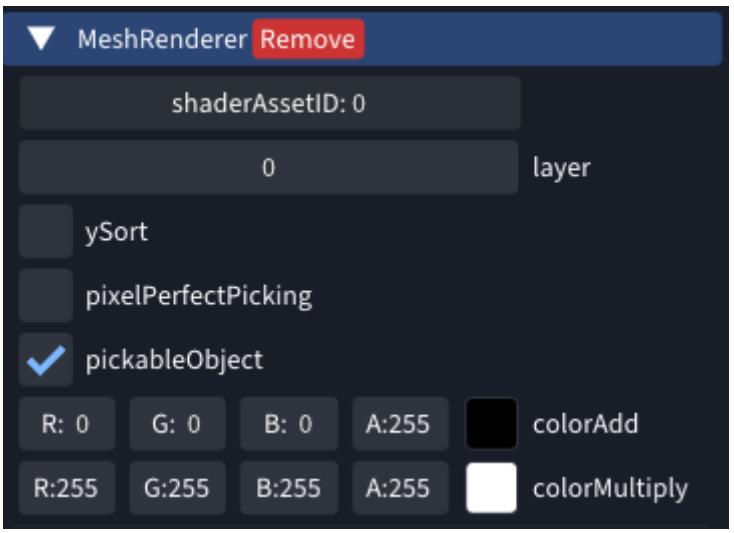
    - Purpose: Stores the entity's name for easy identification in the editor and debugging tools.
    - Editable Field:
      - name: a text string that can be changed directly in the Inspector.
    - Usage: Used by the Hierarchy Tab to display entity names.
- ***LocalTransform (Default Given)***
  - 

    - Purpose: Defines the entity's position, scale, and rotation relative to its parent.
    - Editable Fields:
      - position (x, y): entity's 2D location
      - scale (x, y): non-uniform scaling along each axis
      - Rotation: angle in degrees
    - Notes:
      - If the entity is a child, these values are relative to the parent.
      - The WorldTransform (computed automatically) applies parent transformations recursively.

- ***AABB Collider (AABBFloat (Axis-Aligned Bounding Box))***
  - 

    - Purpose: Defines the entity's collision boundaries for physics and spatial detection.
    - Editable Fields:
      - top, bottom, left, right: numeric edge coordinates forming a rectangular/square hitbox.
    - Usage:
      - Used for collision detection and screen boundary clamping.
      - Automatically updated each frame during runtime.
- ***Rigidbody (Rigidbody2DFloat)***
  - 

    - Purpose: Gives the entity basic physics properties such as movement and mass.
    - Editable Fields:
      - vx, vy: current velocity in X and Y directions
    - Usage:
      - Works with AABBFloat to simulate motion and collisions.
- ***Mesh Renderer***

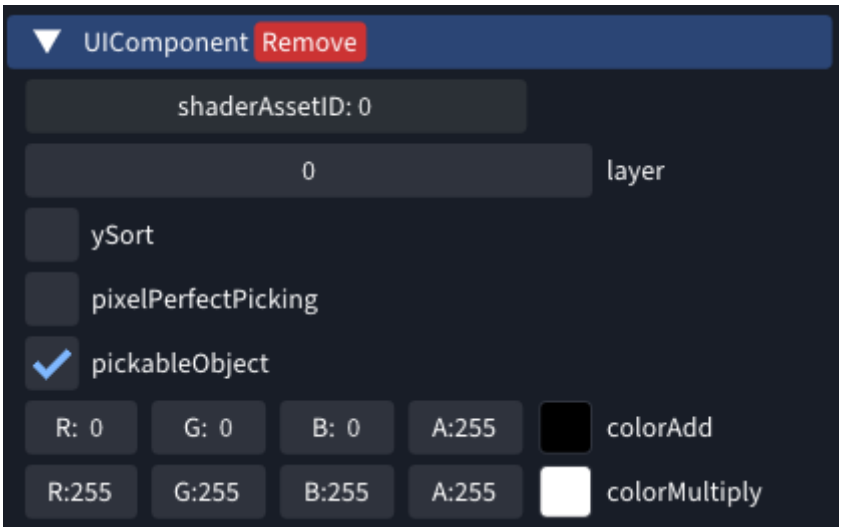


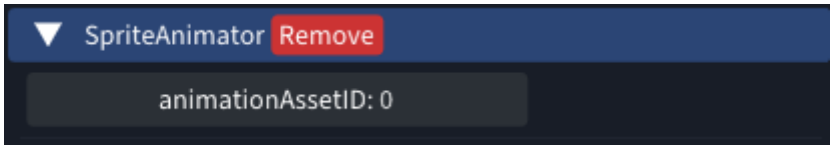
- Purpose: Handles how a mesh is rendered visually on screen.
- Editable Fields:
  - shaderAssetID: references the shader program controlling rendering behaviour
  - Layer: Controls the render order between objects. Objects with higher layer numbers render on top.
  - ySort: Automatically sorts objects based on their Y position.
  - pixelPerfectPicking: Controls how precise mouse picking is. Checks actual pixels. Transparent pixels cannot be clicked.
  - pickable Object: Determines if the object can receive mouse interaction.
  - colorAdd: adds an RGB color overlay (brightening)
  - colorMultiply: multiplies base color for tinting or fading effects
- Usage:
  - Works together with Sprite or mesh data to display textured entities.
  - Allows for lighting, tinting, and transparency adjustments.

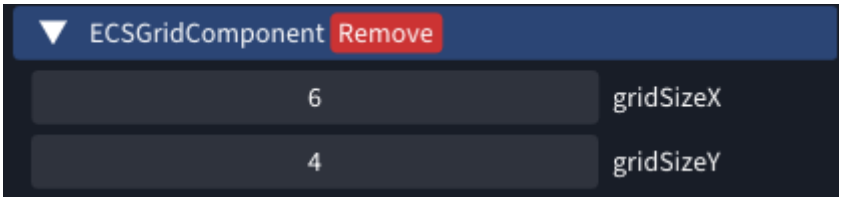
- ***Sprite Renderer (Sprite)***



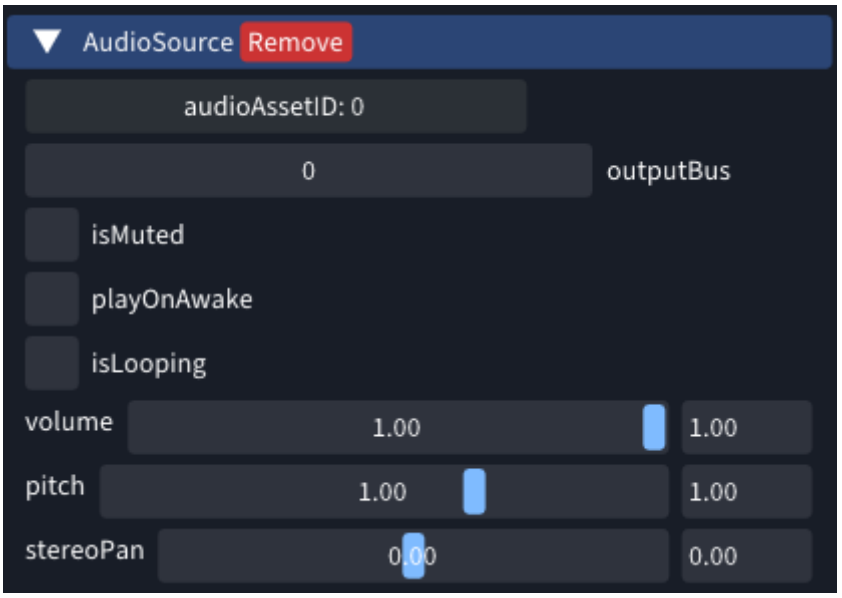
- Purpose: Defines 2D image rendering for sprite-based entities.
- Editable Fields:
  - textureAssetID: the texture or sprite sheet to display
  - textureUVBounds: defines the region of the texture to render (for atlases)
  - textureUVTransform: matrix that controls texture offset, rotation, or scaling
- Usage:
  - Used by the Sprite Rendering System.
  - Allows animated sprites when combined with SpriteAnimator.

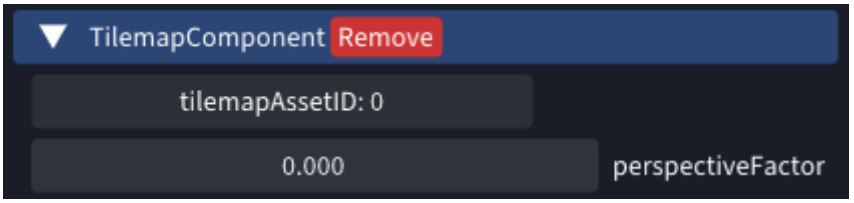
- ***UI Component***
  - 

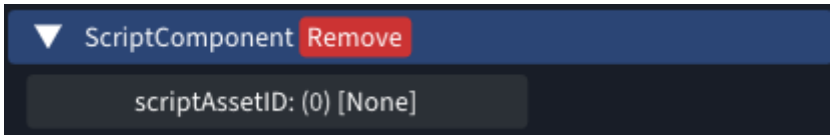
    - Purpose: A lightweight renderer for UI elements such as buttons, panels, icons, sliders, HUD text backgrounds, etc. It functions similarly to MeshRenderer but is optimized for screen-space rendering.
    - Editable Fields:
      - shaderAssetID: shader used specifically for UI .
      - layer: rendering order, higher layers draw on top.
      - ySort: toggle sorting by Y position.
      - pixelPerfectPicking: Controls how precise mouse picking is. Checks actual pixels. Transparent pixels cannot be clicked.
      - pickable Object: Determines if the object can receive mouse interaction.
      - colorAdd: additive color used for hover highlights / glow effects.
      - colorMultiply: multiply tint for dimming or fading.
- ***Sprite Animator***
  - 

    - Purpose: Add sprite-sheet animations such as idle, run, attack, jump.
    - Editable Fields:
      - animationAssetID: the animation data file (.gganim).
- ***Grid Component (ECSGridComponent)***
  - 

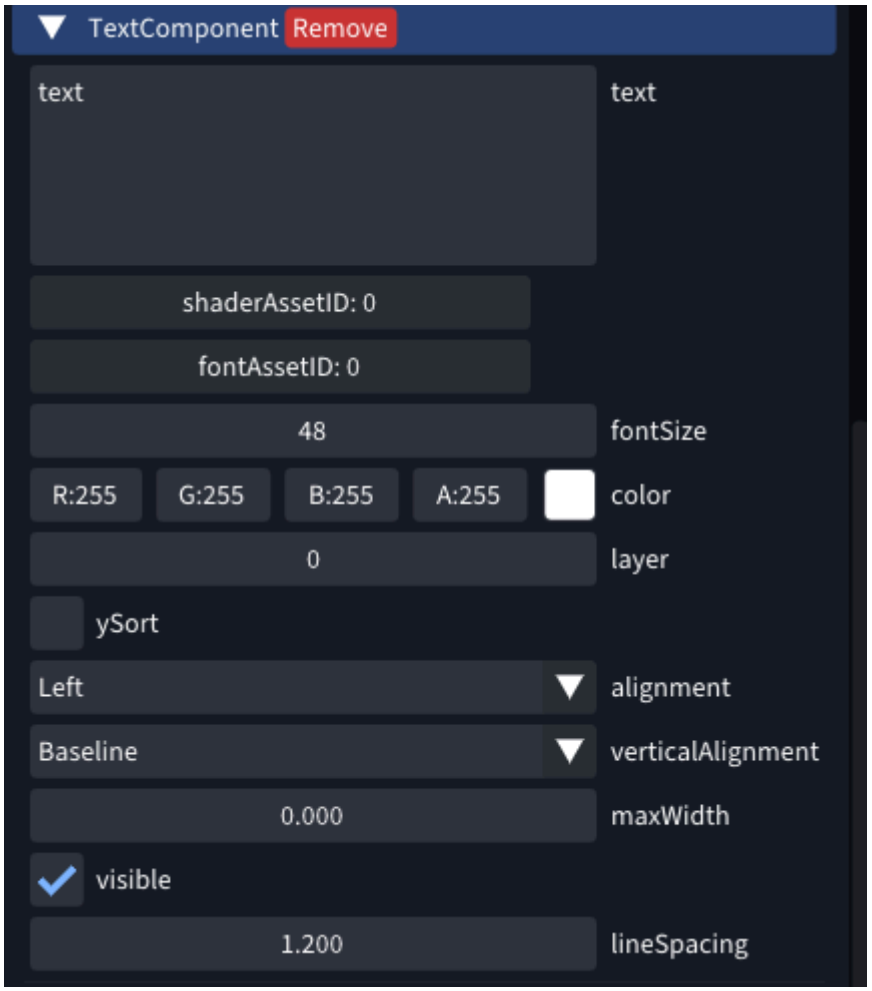
    - Purpose: Defines a grid
    - Editable Fields:
      - gridSizeX, gridSizeY: number of cells horizontally & vertically.

- ***Audio Source***
    - 

    - Purpose: Plays 2D audio with FMOD integration.
    - Editable Fields:
        - audioAssetID: which sound to play.
        - outputBus: audio channel routing.
        - playOnAwake: play automatically when spawned.
        - isLooping: whether the audio loops.
        - isMuted: mute toggle.
        - volume, pitch, stereoPan: playback controls.
    - Usage:
        - Used for sound effects, ambience, and music.
- ***TilemapComponent***
    - 

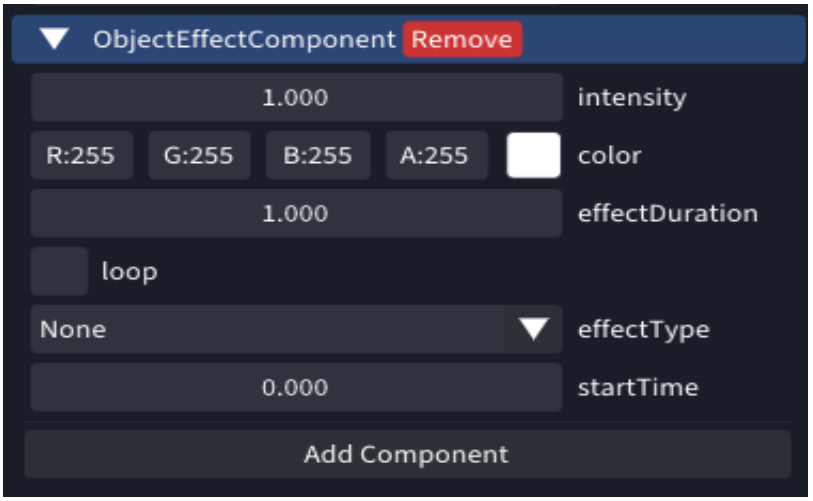
    - Purpose: Loads and renders tilemap assets.
    - Editable Fields:
        - tilemapAssetID: reference to a .ggtilemap asset.
        - perspectiveFactor: applies pseudo-3D skew for isometric effects.
    - Usage:
        - Used to render large tile-based levels efficiently.
- ***Script***
    - 

    - Purpose: Attaches a LUA script to an entity.
    - Editable Fields:
        - scriptAssetID: the script file

- ***Text Component***
  - 

  - Purpose: Renders dynamic text using font assets, suitable for UI elements, labels, and in-scene text.
  - Editable Fields:
    - text: string content to be rendered
    - shaderAssetID: references the shader program used for text rendering
    - fontAssetID: references the font loaded from the Asset Browser (Fonts folder)
    - fontSize: base font size used to generate glyphs
    - color (R, G, B): text color
    - layer: rendering order (higher values render on top)
    - ySort: enables Y-based sorting for text rendering
    - alignment: horizontal text alignment (Left / Center / Right)
    - verticalAlignment: vertical alignment mode (Baseline / Middle / Bottom)
    - maxWidth: maximum pixel width for text wrapping
      - 0 disables wrapping
      - 0 enables wrapping
    - visible: toggles text rendering on or off
    - lineSpacing: spacing in-between lines
  - Usage:
    - Drag font assets from Asset Browser → Fonts into fontAssetID

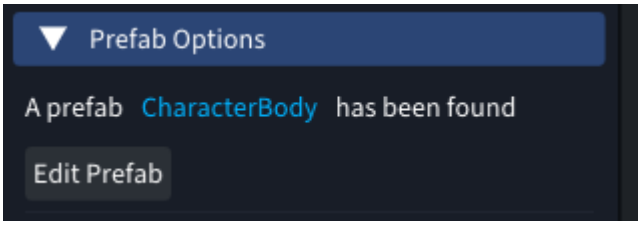
        - Text wrapping is calculated using pixel width, not character count
        - Vertical alignment affects how (x, y) is interpreted:
            - Baseline: (x, y) is the top-left of the text box
            - Middle: (x, y) is the center of the text box
            - Bottom: (x, y) is the bottom-left of the text box
        - Press F8 to toggle the text bounding box outline (debug view)
        - Avoid using many slightly different font sizes; use entity scaling instead to reduce font atlas creation
- ***WorldTextComponent***
    - Purpose: Renders text in world space, meaning the text is positioned using the entity's Transform component and appears as part of the game world.
    - Editable Fields: (same as Text Component)
- **ObjectEffectComponent**



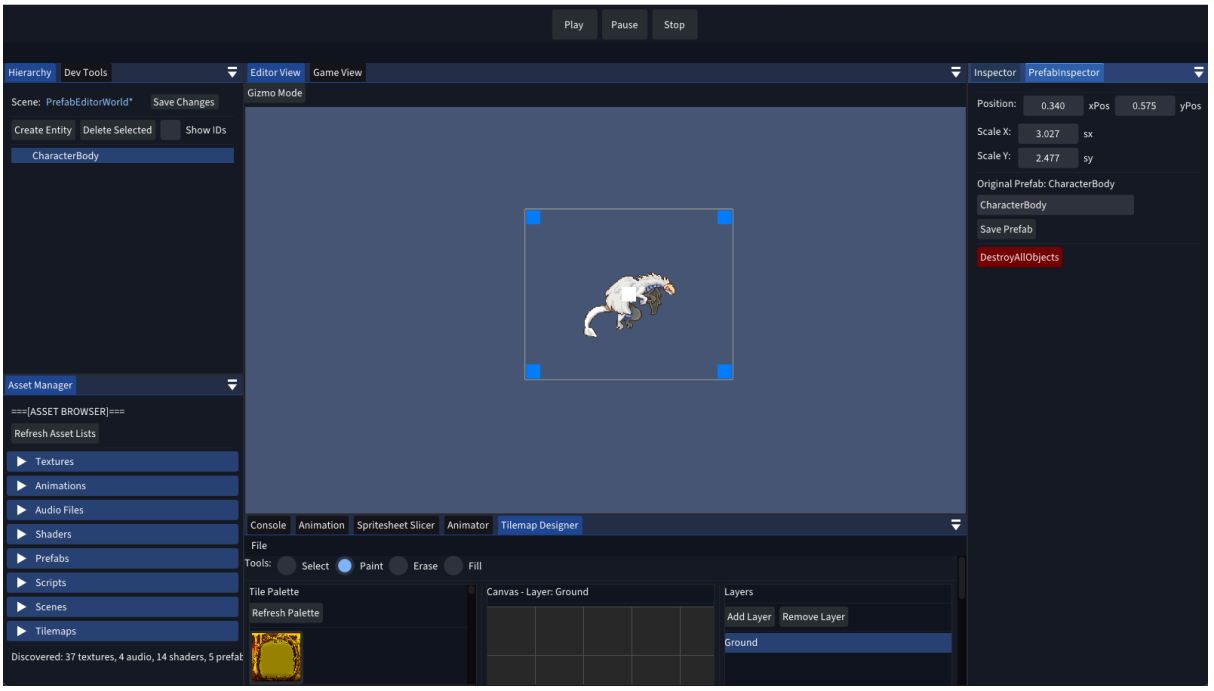
    - Purpose : Applies a visual shader effect to an entity's rendered sprite or mesh over time.
    - Editable Fields:
        - effectType options:
            - **None** - No effect
            - **Fade** - Fades the entity out over the effect duration
            - **Fog** - Blends the entity toward a fog colour over time
            - **Bloom** - Brightens the entity's colours
            - **Shine** - Sweeps a highlight across the entity
            - **Outline** - Draws a coloured outline around the entity's sprite
        - intensity: Controls the strength of the effect (e.g. fade depth, outline thickness, shine width)
        - effectDuration:Time in seconds for the effect to complete
        - loop: When enabled, the effect repeats continuously after completing
        - color: The colour used by the effect (e.g. fog colour, outline colour, shine tint)
    - Usage:
        - Add the component to any entity that also has a MeshRenderer or UIComponent
        - The effect runs automatically once the component is attached and enable loop to see the effect for proper visibility.
        - Start time should not be set the engine still set the start time

6.4 Opening/Editing Prefabs

- Upon selecting an entity in the scene or hierarchy node, if the entity was spawned from a prefab, a prefab option header will be visible in the inspector.



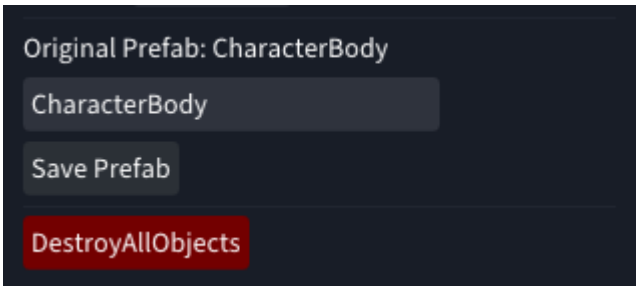
- If the user wants to modify the prefab, they can select “Edit Prefab”, which will bring them into an isolated prefab world for editing.



7.0 PrefabInspector Tab

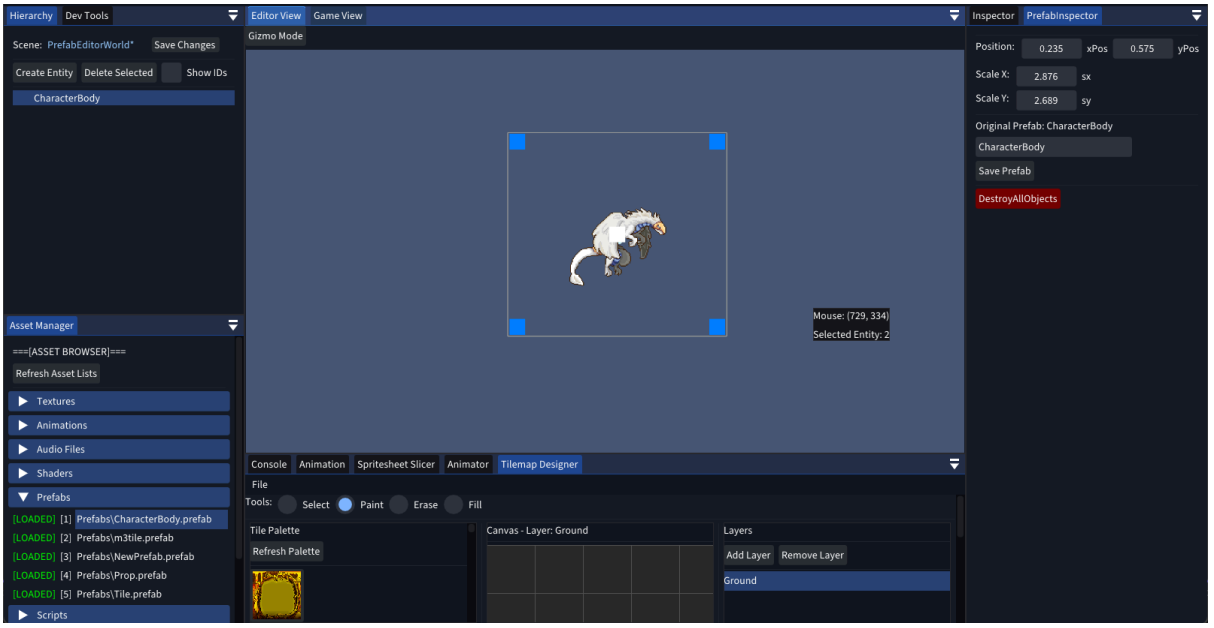
7.1 Editing Prefabs

- Once the user is in the Prefab Editing World, they can click on the Prefab Inspector tab to modify the prefab.
- They can use the sliders to move and scale, or directly right-click and drag to move the prefab around.
- Once done with the modifications, the user can enter a prefab name into the text input box and press save. They can overwrite the current prefab by entering the same name or enter a new name to save it as a new prefab.



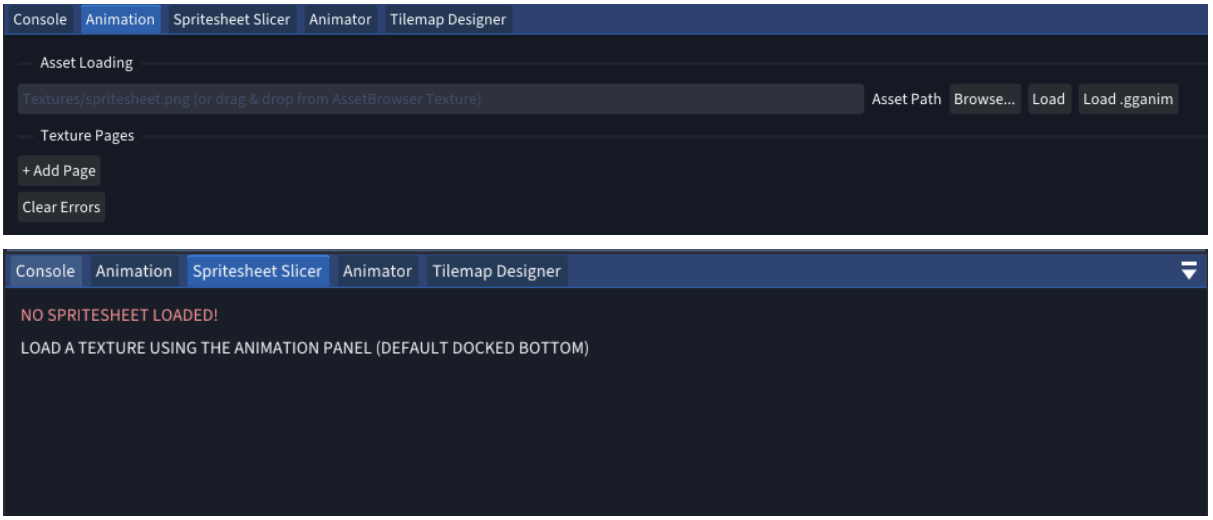
7.2 Dev Tools - Prefab Editor

- The user can also directly edit prefab resources by double-clicking any prefab asset in the Asset Manager. This spawns the prefab directly into the Prefab Editor world.



8.0 Animation Tab / SpriteSheet Splicer Tab

The Animation Tab and the SpriteSheet Slicer Tab work together to let users import textures, slice spritesheets, define animation states, preview animations, and export them into the engine's .gganim animation format.



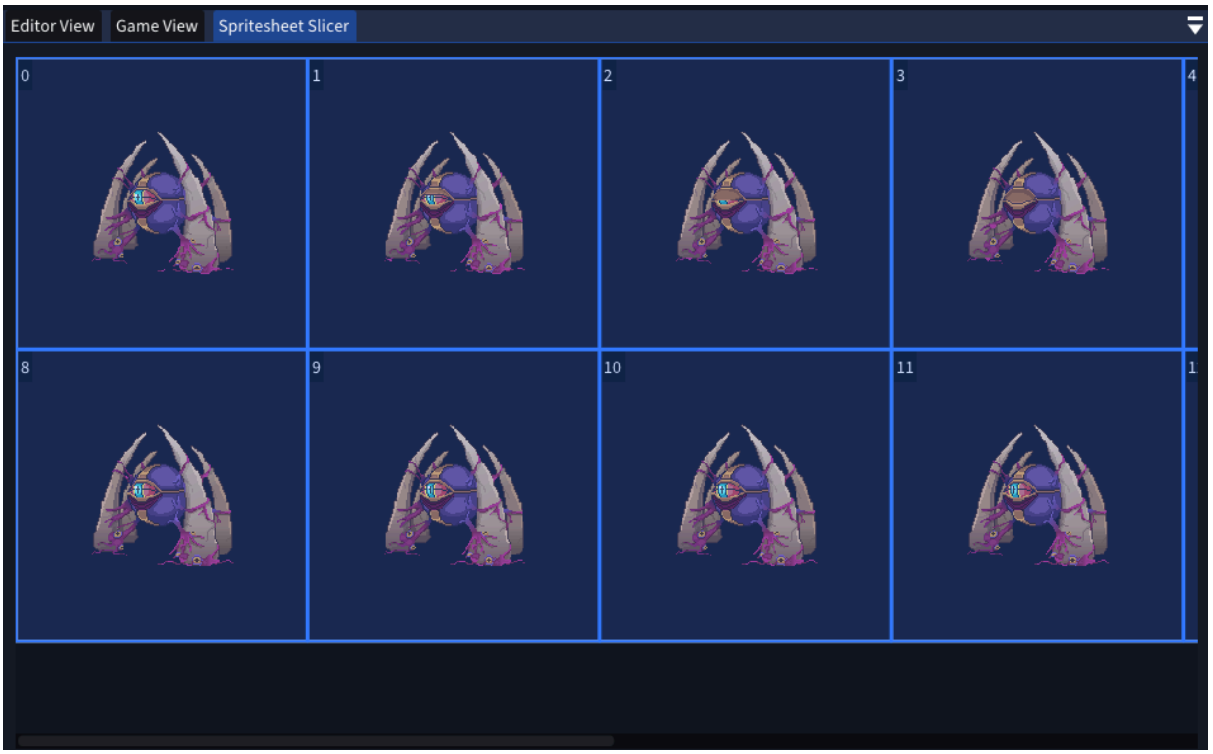
This tool is docked at the bottom of the editor by default and consists of two windows:

- Animation: Contains all controls (loading textures, slicing, animation states, export).
- Spritesheet Slicer: Shows a visual preview of the loaded sprite-sheet with grid overlays and clickable frame-selection.

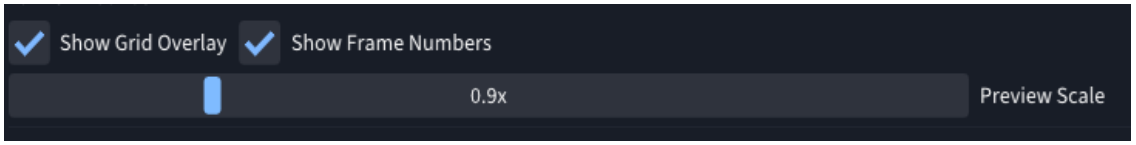
8.1 About Animation (In Animation Tab)

- "+Add Page" (THIS MUST BE ADDED FIRST BEFORE LOADING OR DRAGGING AND DROPPING ASSETS IF YOU ARE LOADING .gganim YOU DON'T NEED TO)
    - Creates a new empty texture page
    - Automatically selects the newly created page as the active page
    - All texture loading, slicing, and animation state editing operate on the active page
    - Add page allows you to add multiple texture to one .gganim(animation)
- Asset Path (e.g. Textures/Spritesheet.png): The editor validates the path, file type, and UUID metadata.
    - Drag & Drop: Drag Texture from AssetBrowser and drop into input Text in Animation.
        - Fills the path input with the dragged Texture
        - Loads the texture into the slicer
    - "Load": Loads the texture currently typed in the input box. Once loaded:
        - The preview appears in the Spritesheet Slicer window
        - Grid settings become editable
- "Browse…": Opens an asset browser popup that lists all discovered texture files.
    - Selecting a file:
        - Fills the path input automatically
        - Loads the texture into the slicer
- "Load .gganim"
    - Opens a popup listing all discovered .gganim animation files.
    - Selecting one:

- Loads the animation metadata (FPS, states, UVs, ranges)
- Automatically loads the associated texture using the stored UUID
- Restores animation states into the editor for further editing

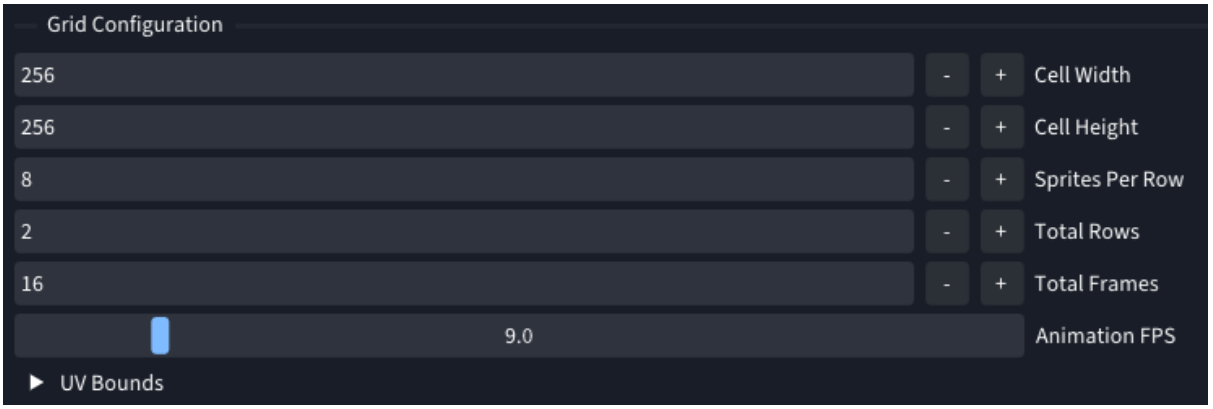


8.2 Spritesheet Slicer Window (Preview Panel)

- When a texture is loaded, the Slicer displays:
- Zooming & Panning
  - Scroll wheel = zoom in/out (0.1× to 4× scale)
- Grid Overlay
  - A visual grid is drawn using the configured cell width/height.
- Options (Available in Animation):
  - Show Grid Overlay (on/off)
  - Show Frame Numbers (on/off)
  - 

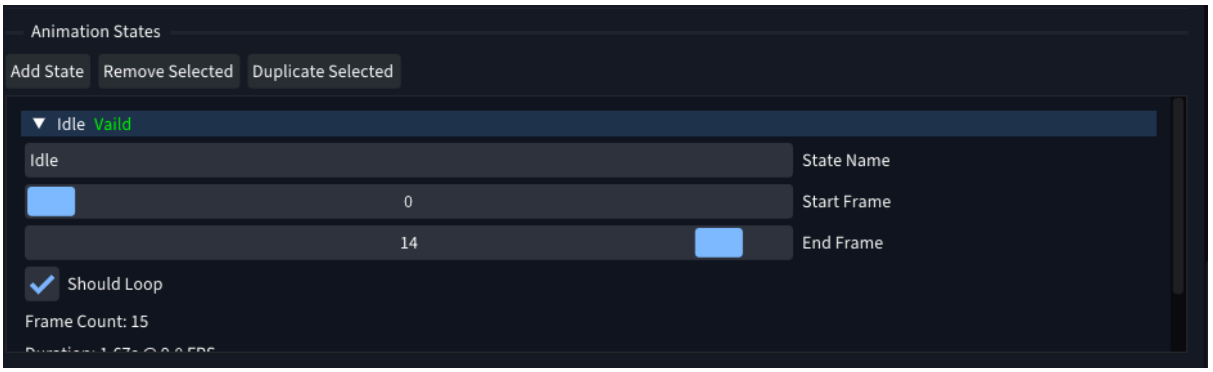
- Clickable Frames
  - Click on any frame in the grid to:
    - Set animation state start/end frames
    - Shift+Click extends an existing range
- State Highlights
  - The currently selected animation state's frames are highlighted in blue and can be viewed in the “Animation Tab” under the “Selected State” Section.
  - 

8.3 Grid Configuration



- The grid settings define how the sheet is sliced into frames.
- Editable (In Animation Tab):
    - Cell Width
    - Cell Height (Sheet automatically calculates: Sprites Per Row, Total Rows, Total Frames)
- Other settings:
    - Animation FPS: playback rate for exported states
    - Preview Scale: visual zoom slider
    - UV Bounds: adjusts texture UV region (advanced use: usually leave default)
        - Contains Reset option
- All changes instantly update the slicer preview.

8.4 Animation States



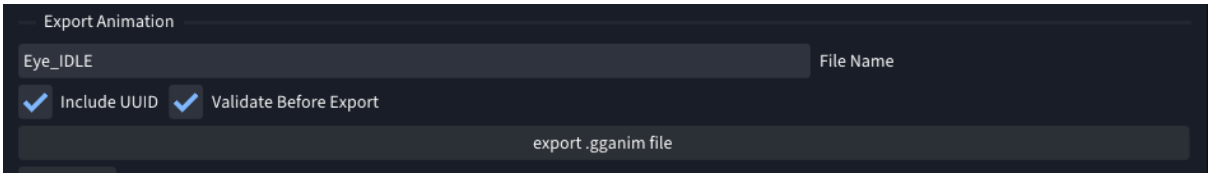




- You may create multiple named animation states (e.g., Idle, Run, Attack).
- State Controls:

        - Add State: creates a new empty state
        - Remove Selected: deletes the selected state
        - Duplicate Selected: copies an existing state
- State List:
    - Each state appears as a collapsible row with:
        - State name, Valid/Invalid indicator, Start Frame, End Frame, Loop toggle, Computed duration & frame count, A small timeline visualizer
- Selecting a state also updates the frame-highlight overlay in the Slicer.
- "+ Add Event"
    - Adds a new animation event to the currently selected animation state
    - A default event named NewEvent is created automatically
    - Each event can be configured with:
        - Event Name - Identifier used at runtime (e.g. for triggering switching of animations)
        - Has Frame (optional)
            - When enabled, the event is bound to a specific frame within the state's frame range
            - When disabled, the event acts as a general state-level trigger
    - Events can be:
        - Renamed
        - Assigned or unassigned a frame
        - Removed individually using the X button

8.5 Exporting Animation (.gganim)



- Export Settings
    - File Name: name of the .gganim file
    - Include UUID: embeds the texture's UUID into the file
    - Validate Before Export: runs validation before saving
- Export Button
    - Only enabled when:
        - A texture is loaded
        - At least one valid animation state exists
        - The file name is non-empty
- Export Behaviour
    - Saves the .gganim file into the Assets folder
    - If file already exists: overwrite confirmation popup
    - After exporting: success popup
- Asset Manager automatically refreshes and shows the new file

8.6 Loading Existing .gganim Files

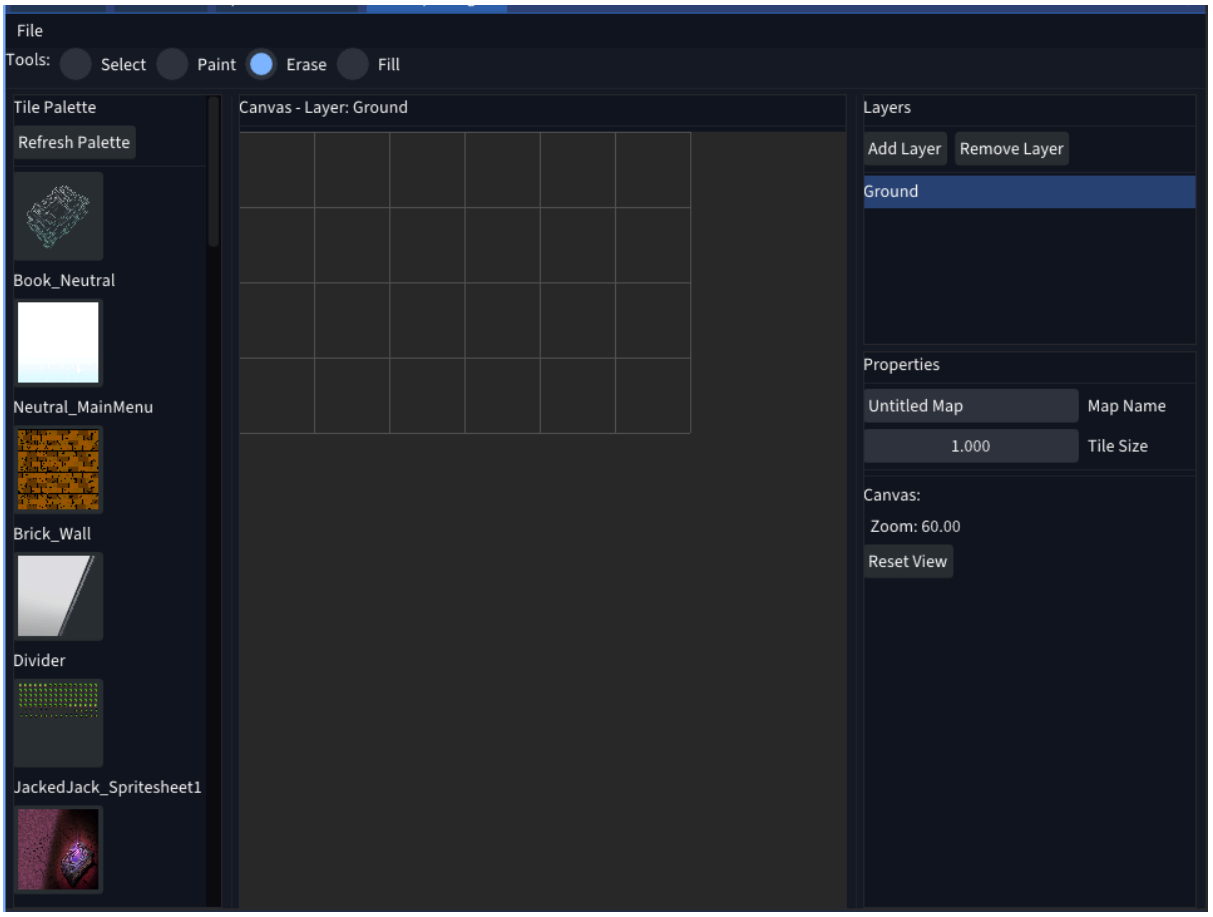
- When selecting a .gganim file from the popup:
    - Texture UUID is read and matched to an existing texture
    - Slicer auto-loads the correct sprite-sheet
    - FPS, UV bounds, sprites-per-row, and all animation states are restored
    - Editor auto-selects the first state
- This lets users modify and re-export already saved animation files seamlessly.

9.0 Tilemap Designer Tab

The Tilemap Designer Tab is a complete visual editor for creating and modifying tactical grid maps (6x4) used by the engine's tilemap system. It provides tools for painting tiles, managing layers, panning/zooming the canvas, inspecting tile properties, validating map structure, and saving/loading .tilemap files.

This tab appears as "Tilemap Designer" in the editor and is usually docked at the bottom.

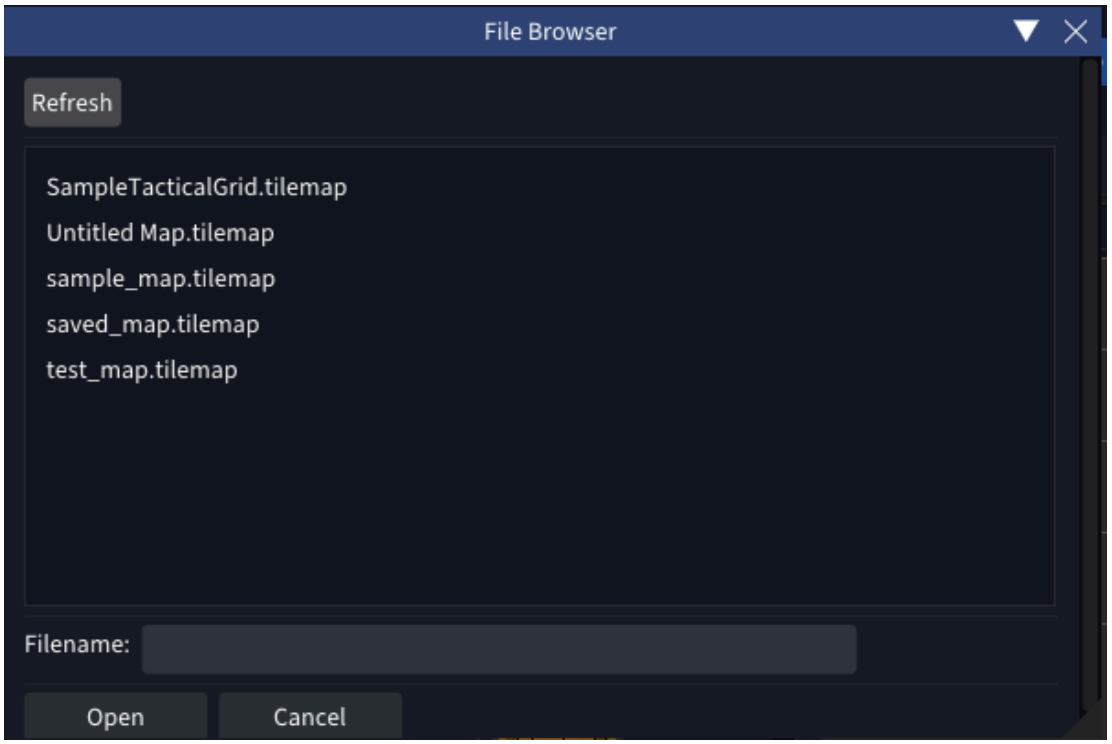
8.1 Overview



- The Tilemap Designer contains three major panels:
- Tile Palette (Left)
    - Displays all available tiles from the Textures/ folder. Clicking on a tile selects it for painting.
- Canvas (Center)
    - Shows a 6×4 grid where tiles can be painted, erased, inspected, or filled.
- Layers & Properties (Right)
    - Manage tilemap layers, visibility, locking, renaming, and view/edit properties for the map and currently selected tile.
- The top of the window includes a File menu (New, Open, Save, Save As) and a Tool Bar (Select, Paint, Erase, Fill).

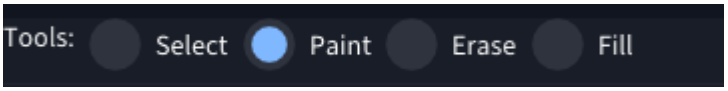
9.2 File Menu (New / Open / Save / Save As) (At the top of the panel)



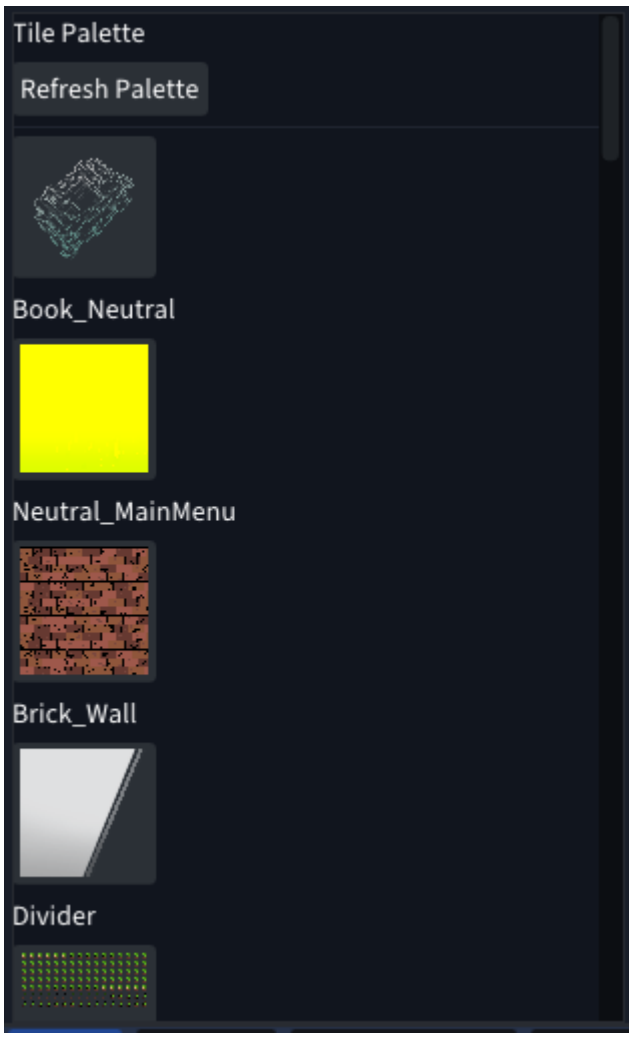
- New
  - Creates a completely new 6×4 tilemap with one layer.
- Open
  - 

  - Opens a file dialog to select an existing .tilemap file.
- Save
  - Saves the current tilemap to its existing filepath.
- Save As
  - Opens a save dialog to choose a new filename for the current map.
  - After saving, the new file appears instantly in the Asset Browser via hot-reload.

9.3 Tools (Select / Paint / Erase / Fill)



- Select Tool
  - Click a tile: highlights it in yellow
  - Tile properties appear in the right-side Properties panel (sprite path, tile type, destructibility, variation, thumbnail)

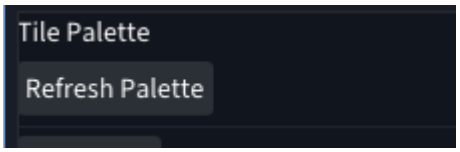
- 

- Paint Tool
  - Left-click paints the selected tile from the palette
  - Supports click-drag painting
  - Automatically switches to this tool when selecting a tile from the palette
  - 

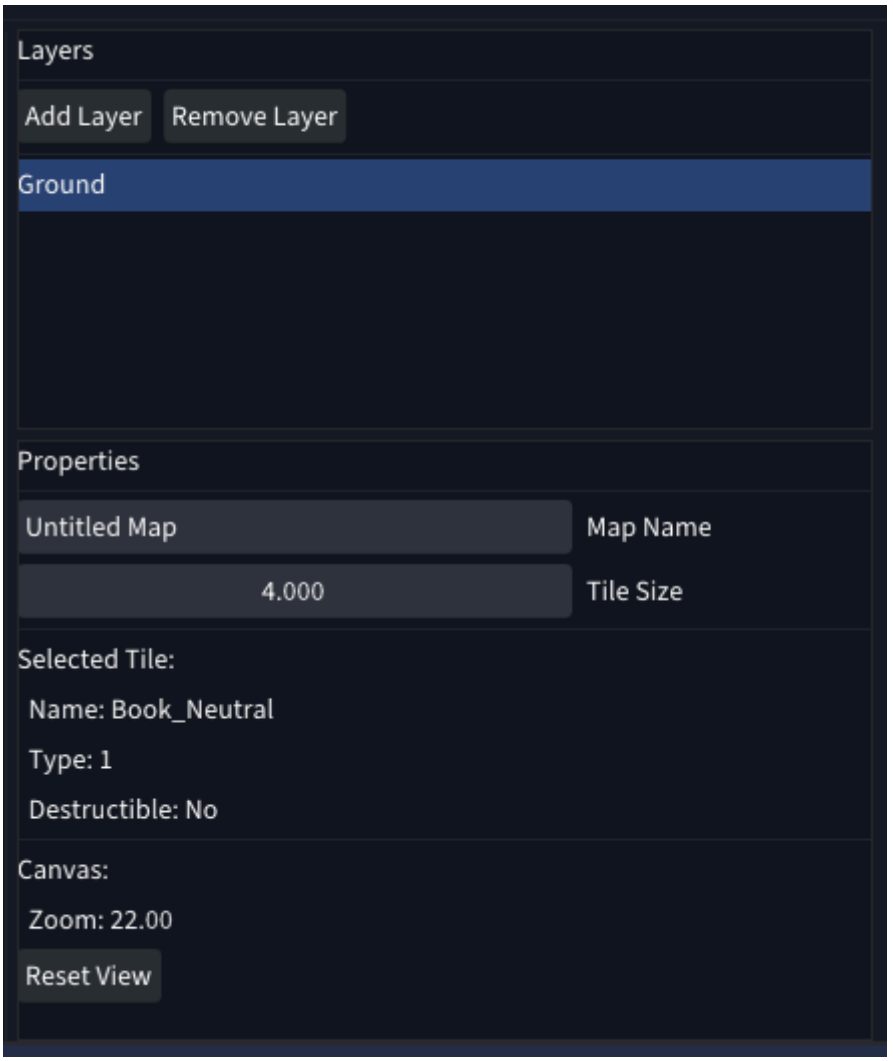
- Erase Tool
    - Removes tiles (resets to empty)
    - Works with both click and click-drag
- Fill Tool (Flood Fill)
    - Performs a flood fill on connected tiles that share the same sprite
    - Replaces all matching neighbors

9.4 Tile Palette Panel (Left)

- The palette shows all textures discovered from the Textures/ directory.
- Features
    - Refresh Palette rescans all loaded textures from the Asset Manager
    - Each tile displays a 64×64 thumbnail
    - Hovering a tile shows:
        - 

        - Tile type
        - Destructible flag
- Clicking a tile selects it for painting
- The selected tile is highlighted
- Click Refresh Palette to Queries AssetManager for all loaded textures
- 

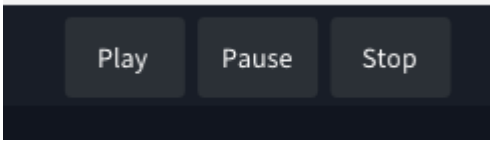
9.5 Properties Panel (Right)

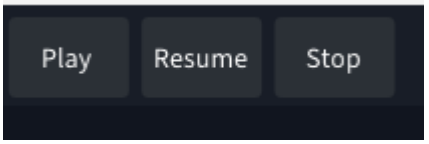


- Tilemap Properties
    - Map Name: editable
    - Tile Size: modifies the visual size of tiles on the canvas
    - Reset View: resets canvas panning and zoom
- Selected Tile (Paint Tool)
    - When a tile from the palette is selected:
        - Display name
        - Tile type
        - Destructible flag

10.0 Editor view / Game view

10.1 Play / Pause / Stop / Resume Toolbar

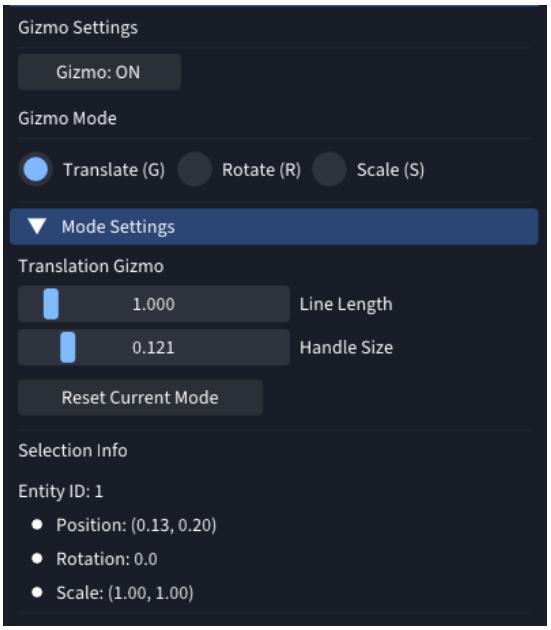




- The Play Mode Toolbar is located at the top of the editor window and controls whether the GGEngine runs in the Editor Mode or Simulation/Play Mode. This allows creators to test gameplay inside the editor without leaving the environment.
- The toolbar includes three main buttons:
    - Play, Pause / Resume, Stop
- The toolbar also automatically handles:
    - Undo/Redo disabling during Play Mode
    - Scene backup + restoration
    - Switching between Editor View and Game View
    - Emitting editor + global playmode events
- Click Play to enter Simulation Mode.
    - The editor switches focus to Game View
    - Scene manager backs up all active scenes
    - SimulationManager begins updating gameplay systems
    - Undo/Redo is disabled
    - An event onStartSimulation is broadcast
    - A global event onPlayModeStart is emitted
    - Renderer switches to gameView mode
- The Play button is automatically disabled while already playing.
- Pause / Resume (When the simulation is running, the middle button toggles between:)
    - Pause
        - Stops simulation updates but keeps the game world in memory.
        - Paused mode is useful for:
            - Inspecting physics states
            - Debugging object positions
            - Stepping through logic
            - Making adjustments before resuming
    - Resume
        - Resumes simulation from the exact point it was paused.
- Click Stop to leave Simulation Mode and return to Edit Mode.
    - Simulation immediately ends
    - Editor switches back to Editor View
    - All scenes are restored from the backups (undoing runtime changes)
    - Any runtime-only objects are removed
    - A onStopSimulation editor event is emitted
    - A onStopPlayMode global event is emitted
- Stopping resets the world to the exact state it was in before pressing Play

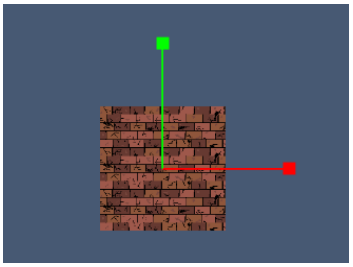
10.2 Gizmos

Camera Settings accessible on the top left “Dev Tools” ⟶ “GizmoSettings”. The Gizmos in the “Editor View” are visual handles that allow users to manipulate an entity’s position directly within the scene, providing a way to move objects without manually entering numerical values in the Inspector Tab.



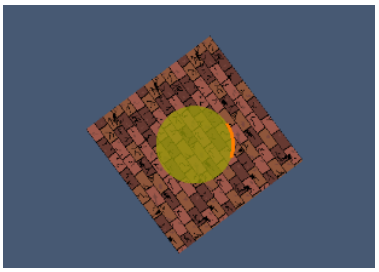
- Right-click on an entity in the editor for Gizmo to appear
- Gizmo can be toggle ON or OFF with Hotkey H or clicking the button under Gizmo Settings
- The movement is applied directly to the entity’s LocalTransform component, and the updated position can be seen in the “Inspector” Tab.
- Notes:
    - The Gizmo only responds when you right-click directly on the colored handle in the mesh.

- Translate (hotkey G)



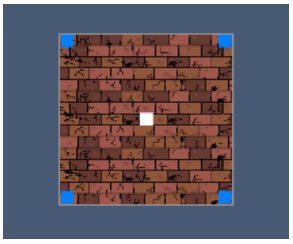
    - The red arrow represents the X-axis (left-right).
    - The green arrow represents the Y-axis (up-down).
    - Hold right-click on the red line within the entity to move it horizontally along the X-axis.
    - Hold right-click on the green line within the entity to move it vertically along the Y-axis.

- Rotate (hotkey R)



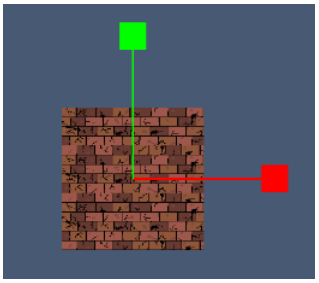
  - The yellow circle represents that the entity is in rotation mode.
  - The orange outline represents the amount of rotation the entity rotated while holding the right click.

- Scale (hotkey S)

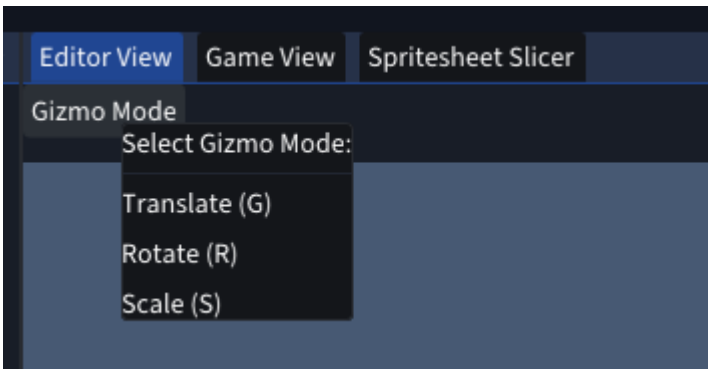


  - The resize handler represents that the entity is in Scale mode.
  - Holding right-click on the blue square can scale the entity bigger or smaller
  - Holding right-click on the white square can uniformly scale the entity

- Mode Settings



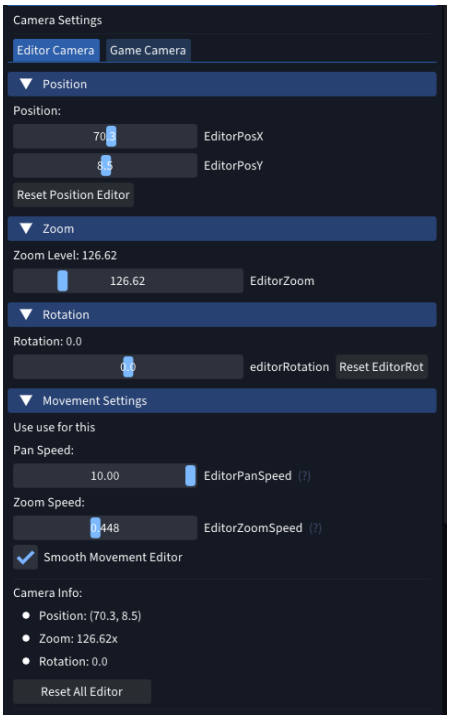
  - Every Gizmo Mode (Translate, Rotate, Scale) have their own mode settings
    - Translate
      - Line Length
      - Handle size
    - Rotate
      - Circle Radius
    - Scale
      - Handle size

- 

    - Gizmo is available below the “Editor View” Tab.

10.3 Camera

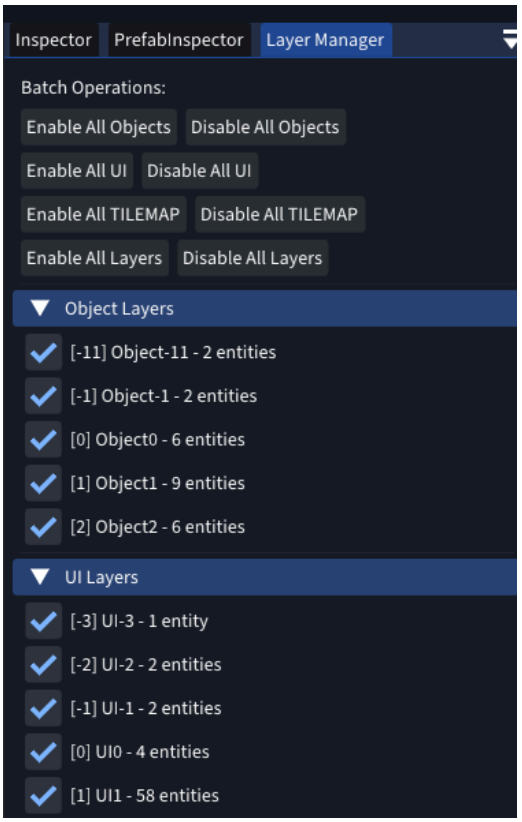
Camera Settings accessible on the top left “Dev Tools” ⟶ “CameraSettings”. The camera component determines what the player or editor viewport sees. It includes settings for position, rotation, field of view, near/far clipping planes, and viewport size. Adjust these values to control the framing, zoom level, and rendering behavior for scenes. The camera can also be moved using editor navigation controls.



- Editor Camera/ Game Camera tab
    - Editor Camera is to change Camera settings in Editor View
    - Game Camera is to change Camera settings in Game View during Play mode
- Position
    - Position can be edited by dragging the EditorPosX or EditorPosY slider
    - Position can be reset to (0, 0) by clicking the “Reset Position Editor” button or pressing the Hotkey Q.
- Zoom
    - Change the Zoom level by dragging the EditorZoom slider
- Rotation
    - Rotate the camera clockwise by sliding to the right
    - Rotate the camera anti-clockwise by sliding to the left
    - Reset the rotation back to 0 by pressing the “Reset EditorRot”
- Movement Settings
    - Increasing the Pan Speed will increase the movement sensitivity while holding the middle mouse button and sliding in the editor view
    - Increasing the Zoom Speed will increase the zoom sensitivity while scrolling with the middle mouse button
    - Smooth Movement Editor will make the movement non-snappy.
- Reset all Editor buttons
    - Clicking it will reset all the settings in the Camera Settings

11.0 Layer Manager Tab

The Layer Manager Tab provides centralised control over rendering visibility for Object, UI, and Tilemap layers. It allows users to enable or disable layers dynamically without modifying entity data, making it useful for debugging, scene organisation, and visual isolation. The Layer Manager reflects the current ECS world state and updates automatically when entities or components change.



10.1 Layer Types

The Layer Manager categorises layers into three types:

- Object Layers
  - Derived from MeshRenderer components
  - Used for world objects such as characters, props, and environment meshes
- UI Layers
  - Derived from UIComponent and TextComponent
  - Used for HUD elements, menus, overlays, and text
- Tilemap Layer
  - Derived from TilemapComponent + ECSGridComponent
  - Controls the visibility of tile-based maps

Each layer displays:

- Layer index
- Layer name
- Number of entities assigned to that layer
- Enable / Disable checkbox

11.2 Enabling and Disabling Layers

Each layer can be toggled individually:

- Tick the checkbox to enable the layer
- Untick the checkbox to disable the layer

When a layer is disabled:

- All entities belonging to that layer are excluded from rendering
- Entity data remains unchanged
- Rendering resumes immediately when re-enabled

11.3 Batch Layer Operations

The toolbar at the top of the Layer Manager provides batch controls:

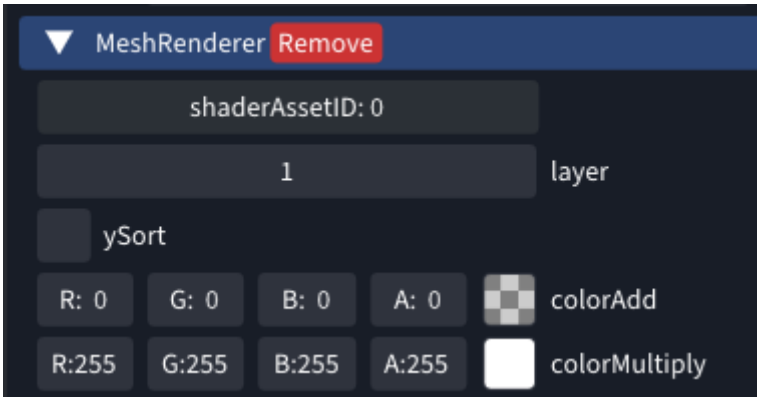
- Enable All Objects / Disable All Objects
- Enable All UI / Disable All UI
- Enable All Tilemap / Disable All Tilemap
- Enable All Layers / Disable All Layers

These operations affect only visibility and do not delete or modify entities.

11.4 Defining Layers in the Inspector Tab

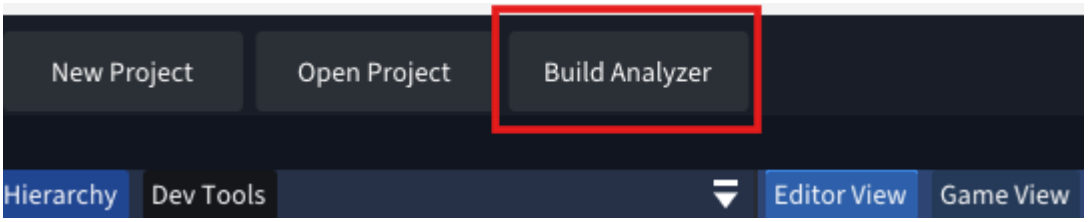
Layers are defined per component in the Inspector Tab, not inside the Layer Manager itself. The Layer Manager reads these values automatically from components attached to entities.

- MeshRenderer, Tilemap, UI and text component
    - Location: Inspector Tab
    - Editable Field: layer (integer)
    - Purpose:
        - Determines which Object Layer the entity belongs to
        - Higher layer values render on top of lower ones
    - Behaviour:
        - Changing the layer value immediately updates the Layer Manager
        - Toggling the corresponding Object Layer will show or hide the entity



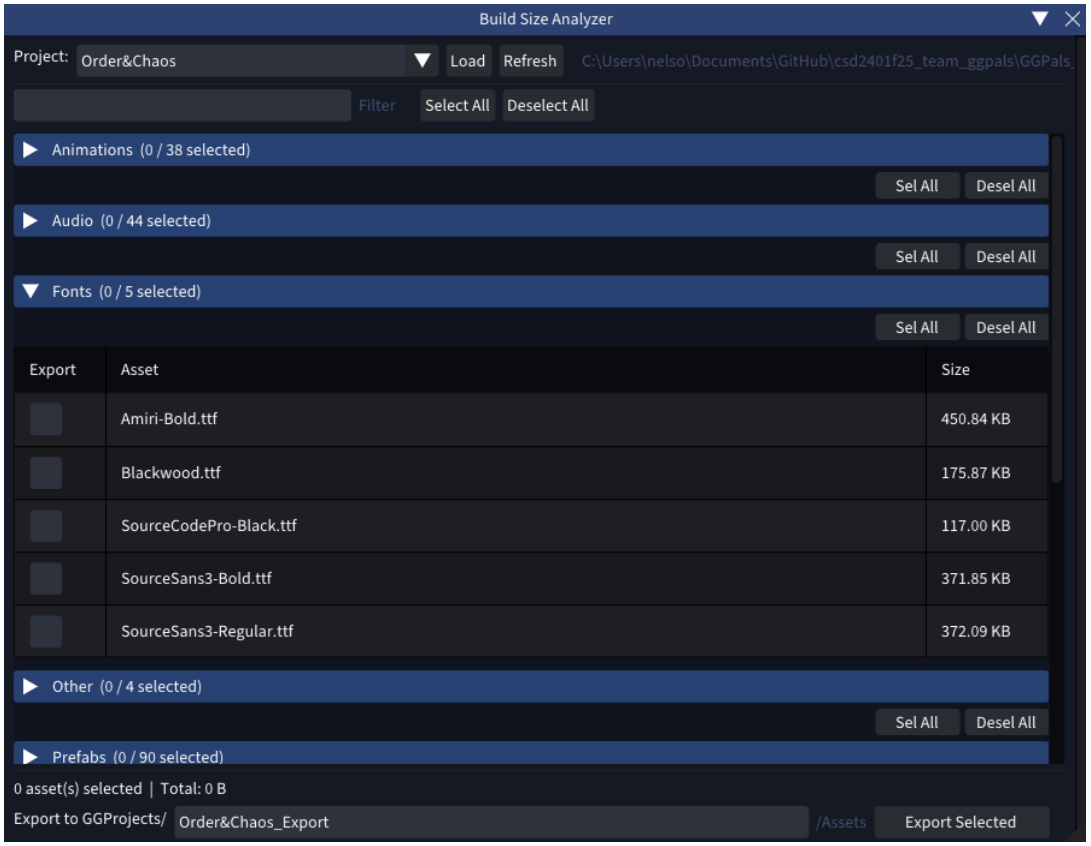
12.0 Build Size Analyzer

Overview

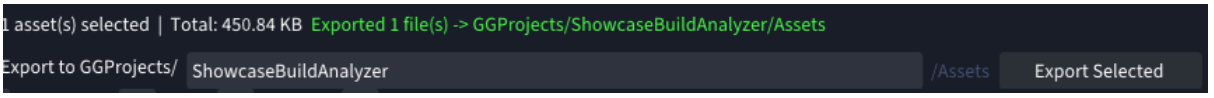


The Build Analyzer button is docked at the top left of the game engine beside Open Project. It has the ability to check selected Assets size, the total selected size and export them to a new project.

12.1 Build Analyzer



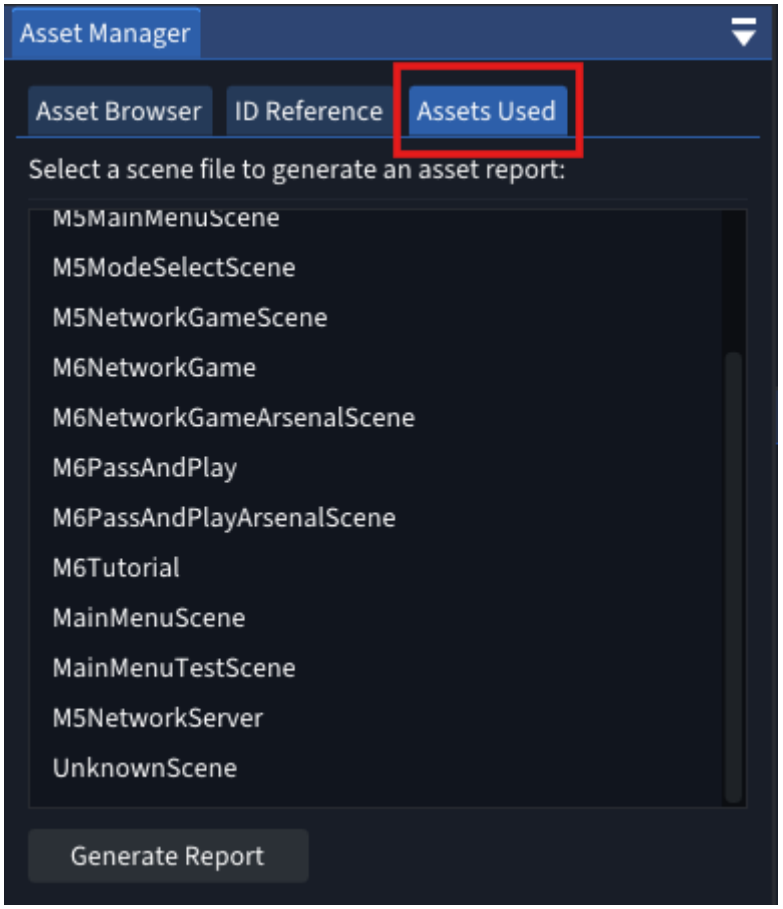
- Select a Project you want to load
- You can use the filter box to find the asset's name you searching for
- You can Select All/Deselect All to check/uncheck all the assets
- All the different assetType are separated with a collapsing header, you can use the Sel All/Desel All to check/uncheck for all the assets within the collapsing head.
- Once you select all the Assets you wanted, you can check the amount of assets selected and the Total size.



- You can change the name of the GameProject name and press "Export Selected" to export all the selected assets into the newly created GameProject

13.0 Asset Used Report

13.1 Asset Used Report



- Asset Used report can be generated by loading a project and in the Asset Manager > Assets Used tab.
- From there select a scene file you want to check asset used
- Hit Generate Report, the txt file will be generated to your game project assets folder

```
================================================================
Assets Used Report: M6PassAndPlay
================================================================

[Textures]  (101 assets)
  Folder: Textures
    - Empty.png

  Folder: Textures/Backgrounds
    - ChaosBoardV1.png
    - OrderBG.png

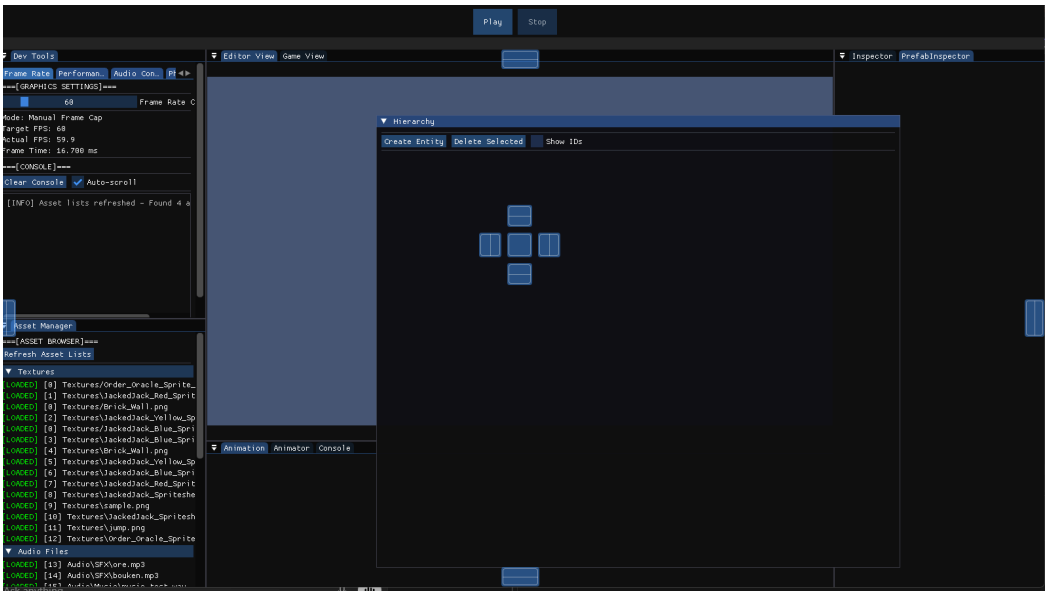
  Folder: Textures/CombatUI
    - EndButton_Empty.png
    - EndButton_Empty_RedEnd-Sheet.png
    - MiddleEndButton_Empty.png
    - Pedestal_Normal.png
    - PointBar.png
    - PointCrystal.png
    - Switch.png

  Folder: Textures/CombatUI/Ability Bar
    - BlackCircle.png
    - Dragoon_IndivFrame.png
    - Eye_IndivFrame.png
    - Imp_IndivFrame.png
    - Oracle_IndivFrame.png
    - Tirion_IndivFrame.png
    - Vizom_IndivFrame.png

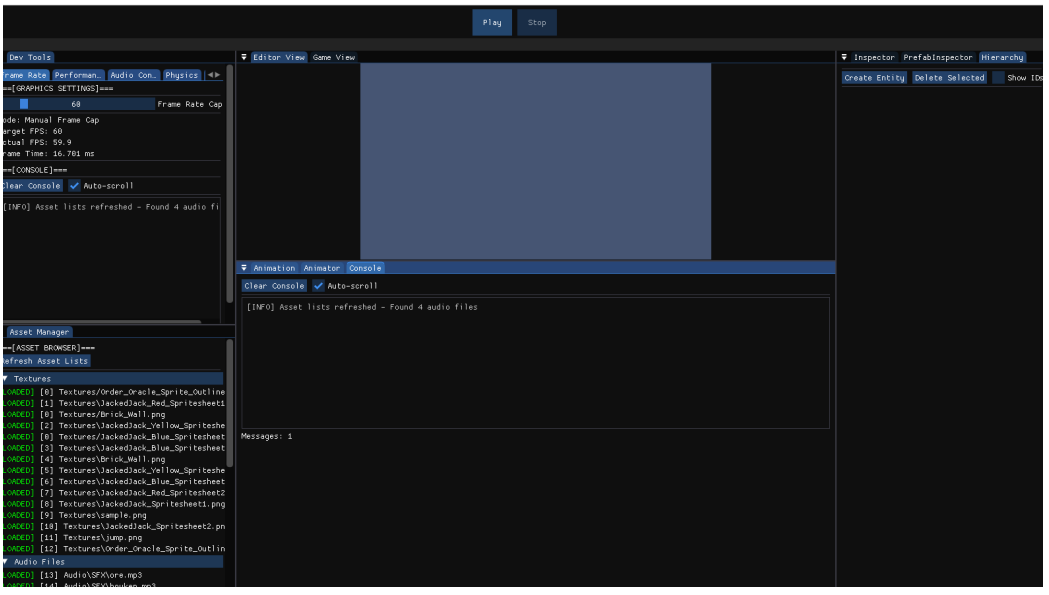
  Folder: Textures/CombatUI/Ability Bar/ExportedFrames
    - Dragoon_IndivFrame.png
    - Eye_IndivFrame_Small.png
    - Imp_IndivFrame_Small.png
```

14.0 Others

14.1 Docking

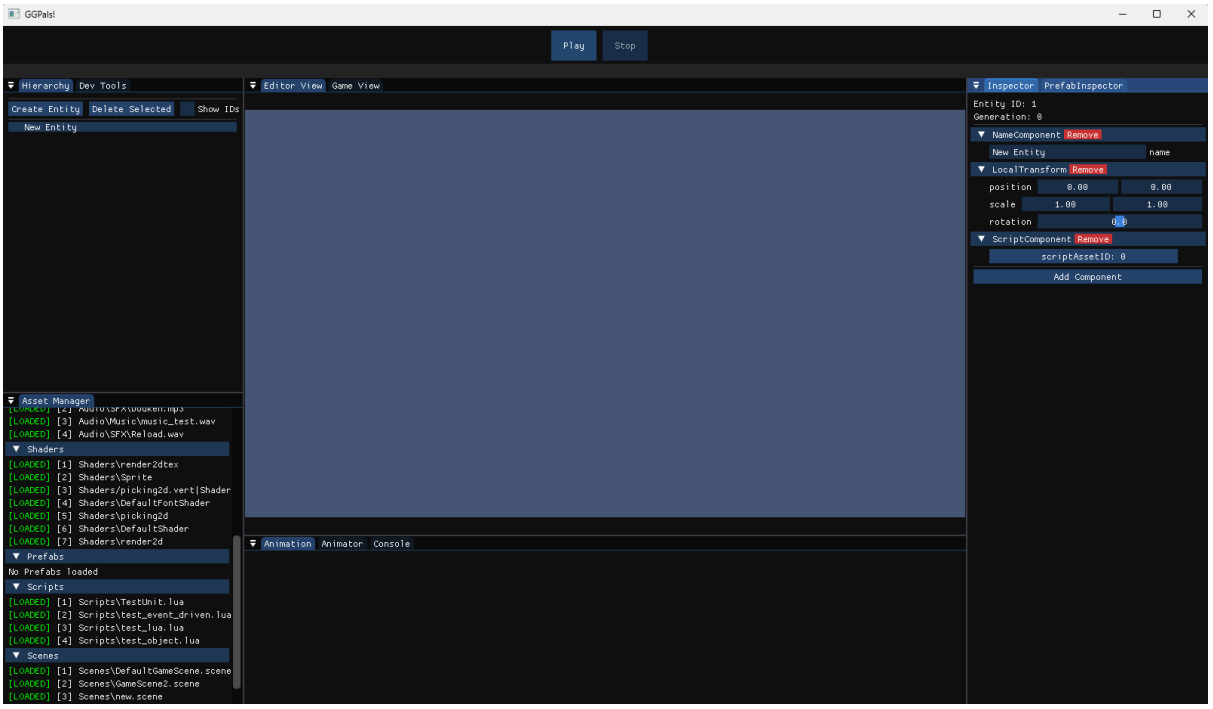


- Left-click and hold the tab header (e.g., Hierarchy, Inspector, Console) and drag it to a new position.
- A blue outline will appear, showing potential docking zones (top, bottom, left, right, or center).
- Release the mouse to dock the tab in the highlighted area.
- Undock a Panel:
- Drag a tab away from the docking area until it becomes a floating window.
- This is useful for multi-monitor setups or for isolating key panels.



- Hover your cursor between two panels until a resize handle (a vertical or horizontal bar) appears.
- Click and drag to resize the panel to your desired width or height.

14.2 Lua/Scripting



- To test Lua Scripts:
    - Step 1: Launch editor
    - Step 2: Go to Hierarchy tab
    - Step 3: Press “Create Entity”
    - Step 4: Go to Inspector tab and add component “Script”
    - Step 5: Go to Asset Browser, and scroll down to "Scripts"
    - Step 6: Select "Scripts\TestUnit.lua", and drag it to Inspector Tab's "scriptAssetID
    - Step 7: Go to Win32 console and observe the script running

14.3 Undo and Redo

- CTRL + Z to Undo action by 1 step
- CTRL + Y to Redo action by 1 step

14.4 FullScreen & Window mode

- Hotkey F11 to toggle FullScreen & Window Mode

14.5 Show all entities on screen

- Select and entity on screen and click Hotkey F to fit all the entities in editor view frame;